EthernaCare

By

Oo Kai Heng

FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY
KUALA LUMPUR

ACADEMIC YEAR
2026/2027

[EthernaCare]

By

[Oo Kai Heng]

Supervisor: Dr. See Kwee Teck

A project report submitted to the
Faculty of Computing and Information Technology
in partial fulfillment of the requirement for the
Bachelor of Software Engineering (Honours)

Faculty of Computing and Information Technology
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.

_____

Student Name:

Oo Kai Heng

ID:

2402185

Abstract

EthernaCare is a Flutter mobile application designed to support elderly people and independent individuals who live alone. It combines a daily virtual-pet check-in, configurable inactivity monitoring, trusted-contact escalation, location-assisted emergency records, rewards, legacy planning, weather-responsive visuals, and AI guidance. The objective is to reduce delayed awareness of prolonged inactivity while providing an accessible way to organize safety information and personal preferences.

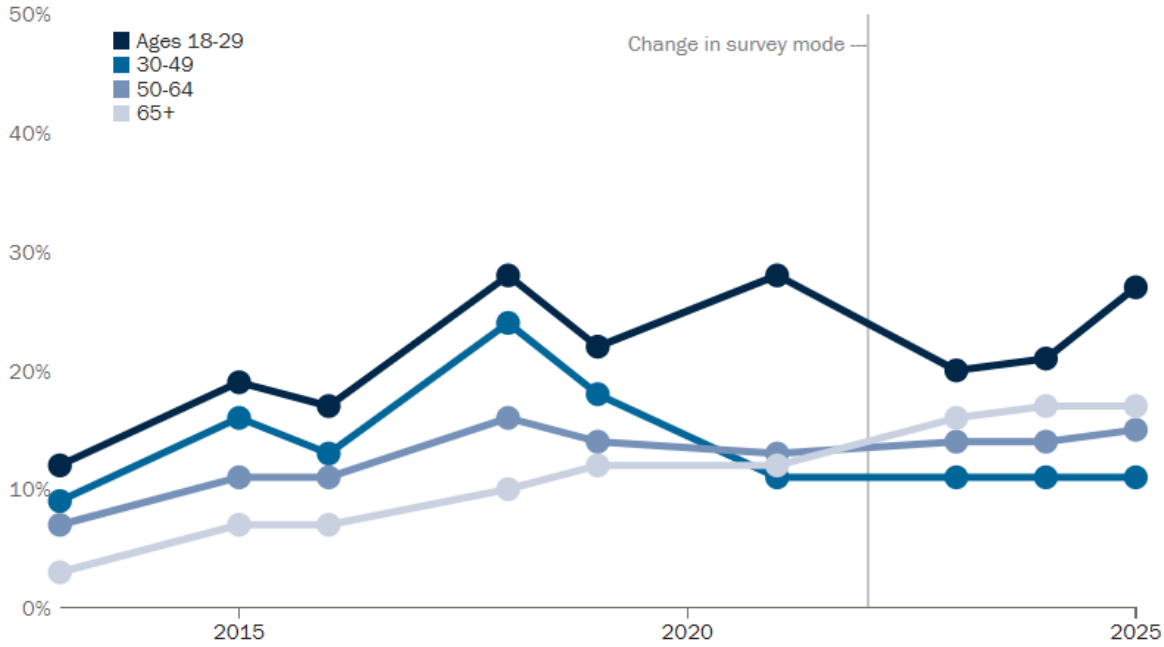
The daily safety signal is completed by tapping Oren, the virtual cat. Each successful daily check-in is stored in Supabase and can award Oren tokens and streak-based rewards. The Android app uses WorkManager for best-effort periodic inactivity checks. A local reminder counter progresses through three missed threshold windows; reminders 1 and 2 prompt the user, while reminder 3 starts the configured escalation. When the primary trusted contact option is selected, the app attempts an automated SMS and records the alert and available location. The app never automatically calls Malaysia's 999 service; a 999 call requires an explicit user action.

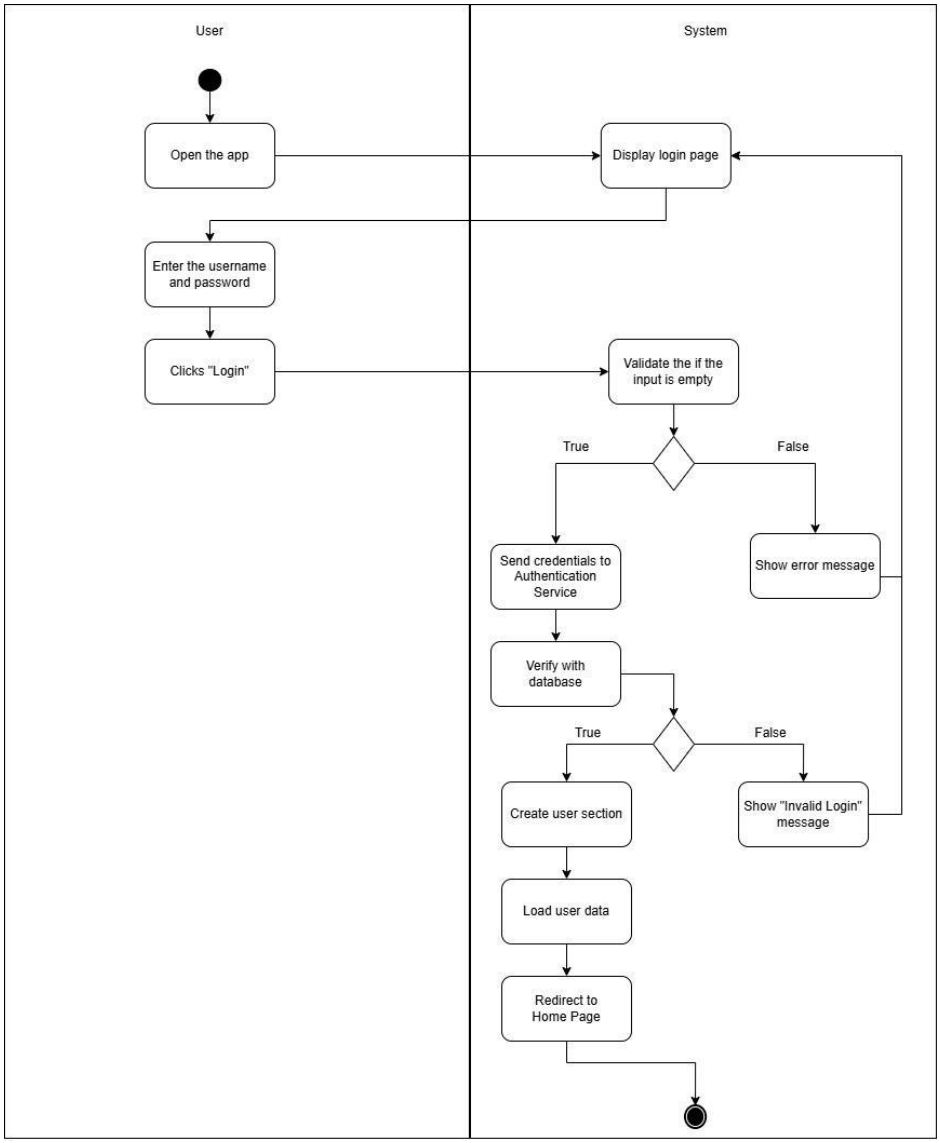
The project follows an Agile approach. Flutter implements the responsive Android and web interface, while Supabase provides authentication, PostgreSQL data storage, Row Level Security, private file storage, and Edge Functions. Malaysia's official weather API is used first, with Open-Meteo as a location-based fallback. Gemini provides server-side AI guidance with a built-in offline fallback. SharedPreferences caches selected dashboard, weather, reward, chat, Oren, and inactivity state to reduce repeated network requests. Automated widget and logic tests verify the principal user flows, while external SMS, OAuth, weather, and AI providers require configured credentials and integration testing.

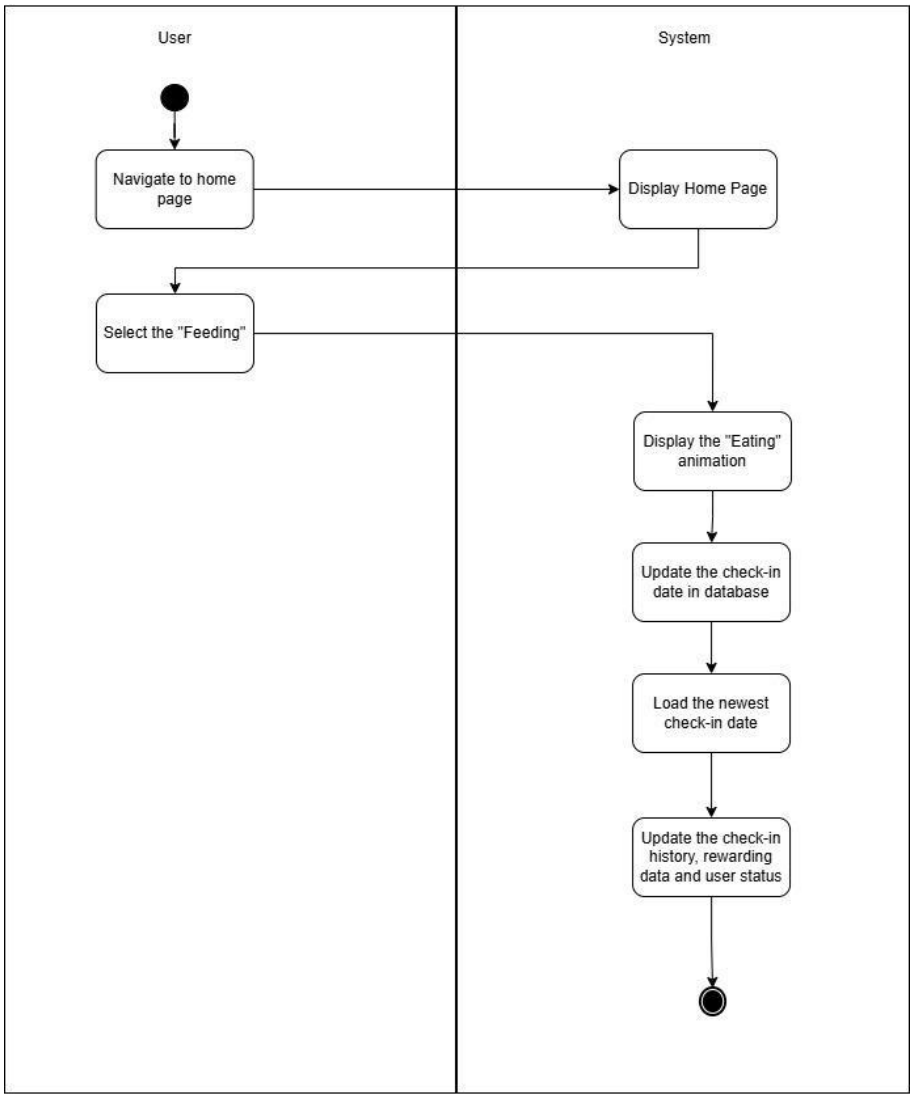
Acknowledgement

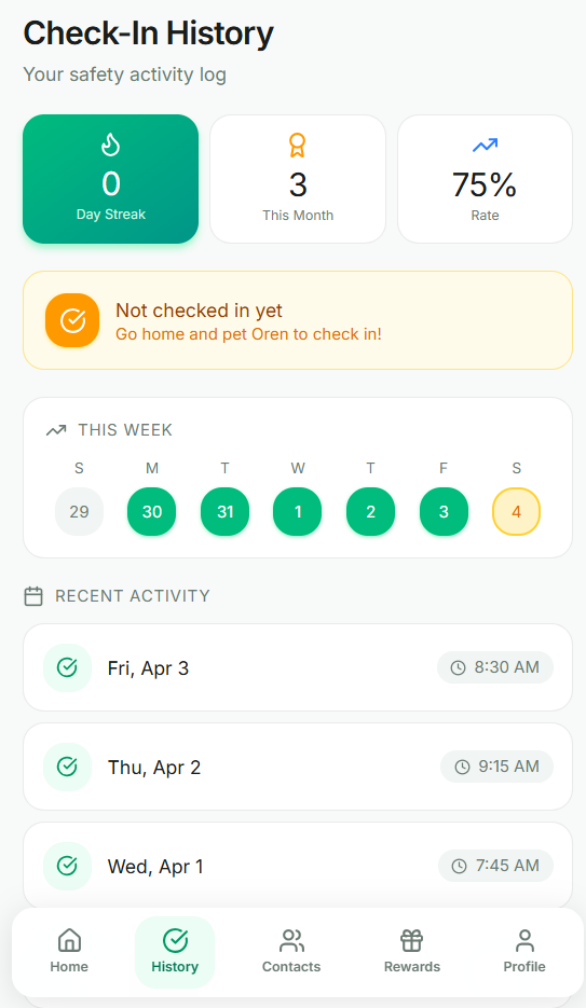
This contains acknowledgement to those who have contributed directly or indirectly to the completion of the project. Usually the people to be acknowledged include the project supervisor(s), moderators, family, and those who have given assistance and supports to ensure the success of the project.

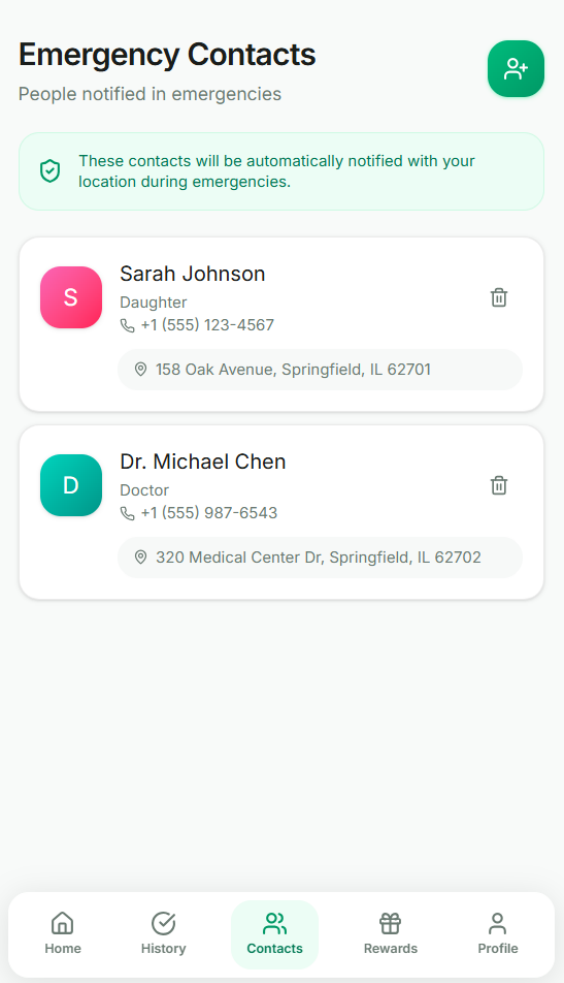
Table of Contents











Chapter 1

Introduction

1 Introduction

This chapter presents the background and motivation of the EthernaCare project. It explains the social problem related to elderly individuals living alone and the risks linked to delayed discovery during emergencies. The chapter also outlines the project objectives, scope, and significance to provide a clear understanding of what the system aims to achieve.

1.1 Project Objective(s)

The project aims to utilize a **gamified interaction** such as **petting a virtual cat** to confirm elderly users are active. Instead of traditional messages, the user interacts with a virtual companion to record a check-in, making the safety verification process more engaging and less clinical.

Another objective of the project is to support elderly users in preparing for post-death matters. This includes allowing users to record their funeral preferences and store digital copies of wills in a secure manner. By providing this function, the project helps reduce confusion, emotional stress, and administrative difficulties for family members after the user's death.

The project also aims to provide basic information support through an AI-assisted chatbot. This feature is designed to help users and family members access general guidance related to funeral arrangements and related matters in a simple and direct way.

Overall, the project fulfills the requirements of "SDG 3 – Good Health and Well-Being" and seeks to provide a practical and accessible solution that improves elderly safety, preserves personal dignity, and supports families by using smartphone technology that users already own.

1.2 Background of the Project

1.2.1 Malaysia is about to become an aging society

According to the prediction from "Portal Rasmi Kementerian Kewangan", Malaysia will become an aged nation in 2048 (Azizan, 2025). The number of elderly people living alone will definitely increase as the elderly population increases. Moreover, many elderly people have poorer health compared to younger people and face a higher risk of death. These conditions increase the chance of dying alone without immediate discovery.

1.2.2 Migration and Elderly Parents Living Alone

In recent years, many young adults have moved from home to other areas for work or education. Internal migration remained dominant at 63.5% in 2024 (up from 62.3% in 2022), Urban to urban migration dominated at 84.6% in 2024 (up from 79.3% in 2022)(*Migration Survey Report, Malaysia, 2024*, 2025). This often happens when the children are grown up and want to start their own lives. As a result, parents are left living alone in their house. The contact and interaction between parents and children becomes less frequent, children are busy in their own business and usually only back to home on certain holidays. When elderly parents live alone, they are facing the higher risks of dying alone because help may not be available immediately. This situation increases social isolation.

1.2.3 Increase in the Number of Single Individuals

The number of marriages decreased 12.5 per cent from 215,022 (2022) to 188,100 (2023). (*Migration Survey Report, Malaysia, 2024*, 2025). In 2020, there were 14.5 male and 14.3 female births per thousand population, a decrease from 15.1 male and 14.9 female births per thousand population in 2019 (*International Journal Of Academic Research In Business & Social Sciences*, 2024). It was the lowest rate recorded in the country. Combined with these factors, the number of single individuals without children will increase in the future. Against the backdrop of soaring consumption levels, many people are facing delayed marriage, financial issues, work pressure or personal choices. Others remain single due to divorce or the loss of a spouse. As a result, more adults live alone for longer periods of time. This change affects daily support systems, especially as people grow older. Decades later, the number of living alone elderly people will inevitably increase.

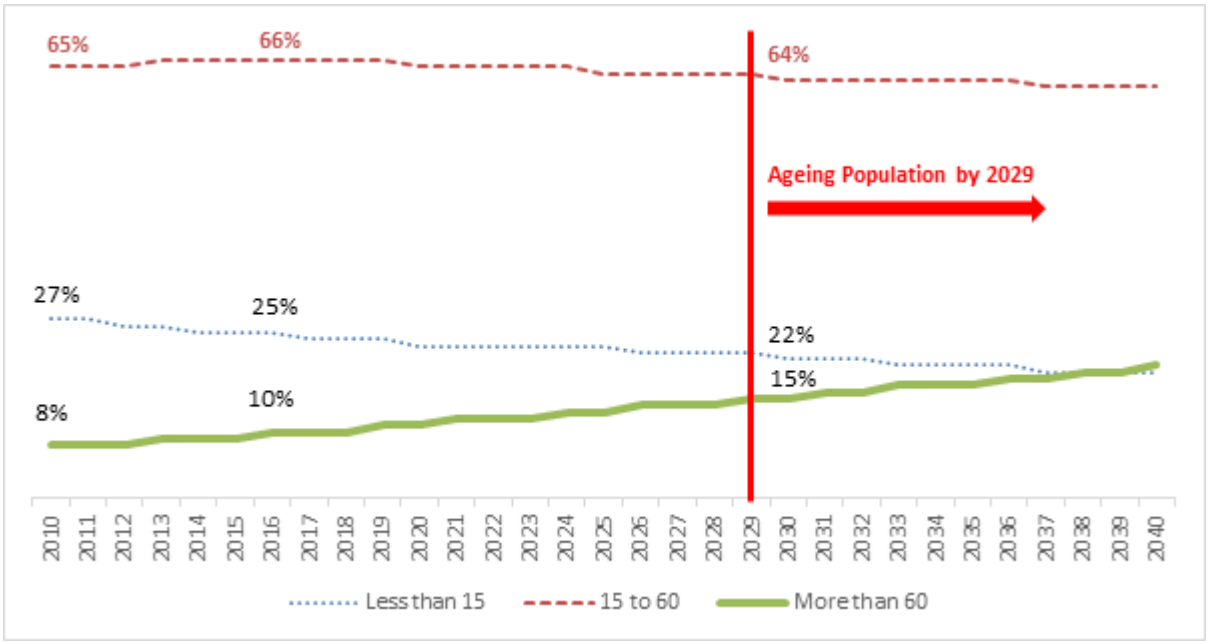


Figure 1.2.1: The prediction of the population age group in the future

(Malaysia Population Research Hub, n.d.)

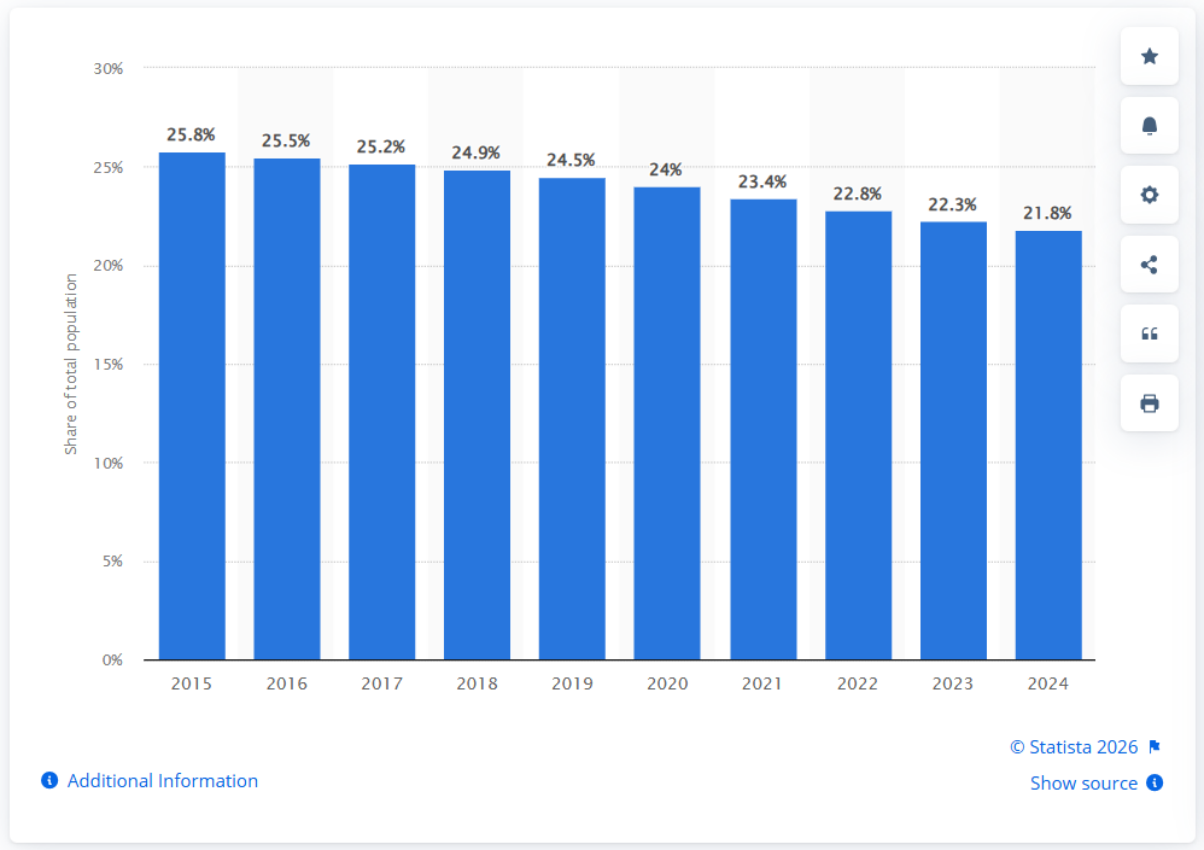


Figure 1.2.2: Children in total population in Malaysia from 2015 to 2024

(Statista, 2025)

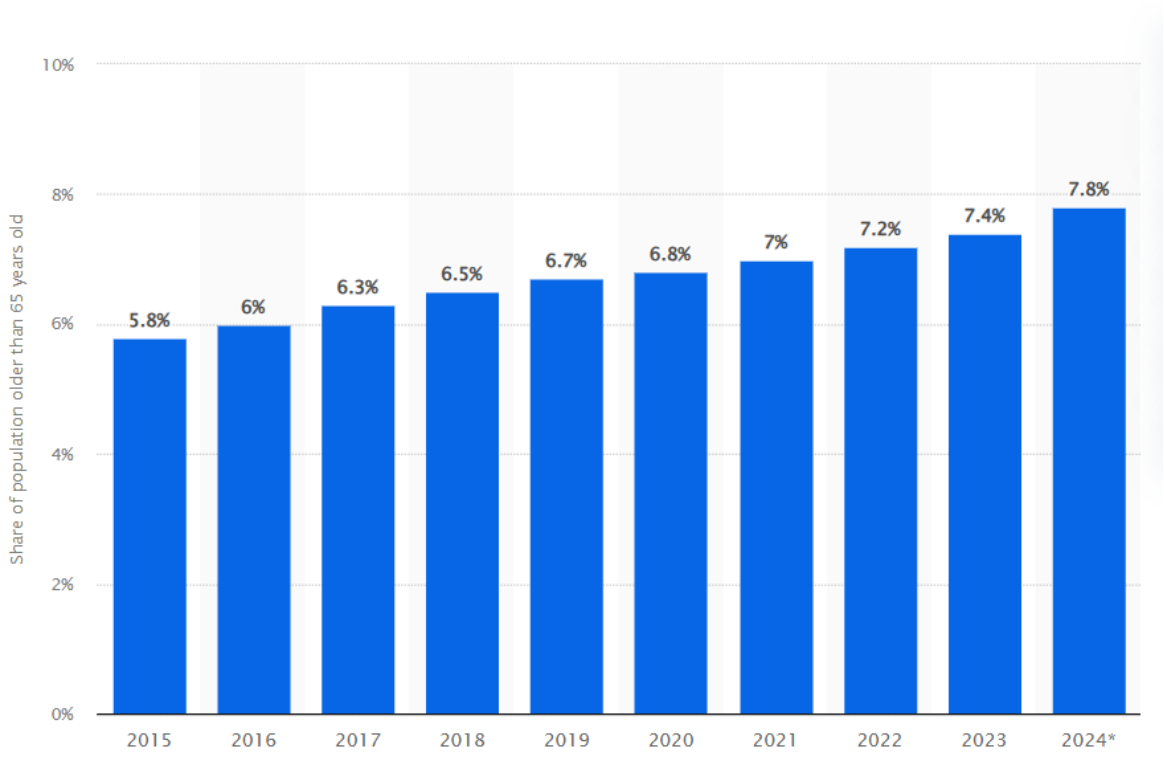


Figure 1.2.2: Aging population Malaysia 2015-2024

(Statista, 2025)

1.3 Advantages & Contributions

1.3.1 Easy to access and achieve

The project provides a solution that does not require extra devices or complicated devices. It relies on the smartphones that are owned by the users. The system uses a **gamified pet-care interaction** that encourages regular use. By transforming the check-in into the act of 'caring' for a virtual cat, the system reduces the 'surveillance' feel of the app and replaces it with a sense of companionship and purpose.

1.3.2 Reducing the unfortunate cases & Notified families in time

The project contributes by reducing delays in emergency response and death discovery through **daily check-in verification**. Once the user response is out of time, user families will auto receive the notification. Even though the elderly user cannot be rescued in time, at least can be found in time to prevent the body from rotting.

1.4 Project Plan

The project follows an Agile mobile application development approach. Development is divided into several phases that include planning, system design, implementation, testing, and deployment. Core features such as activity detection and emergency alert notification are developed early in the project. Features related to funeral preferences, will management, and AI-assisted guidance are developed in later stages. Testing is carried out throughout the development process to ensure that each function works correctly and meets user needs.

Project Schedule:

ACTIVITIES	EXPECTED OUTCOME	COMPLETION DATE
Proposal	Determine the objective, problems and solutions.	16/12/2025
Phase 2: FYP1		
Chapter 1: Introduction	Introduce the project purpose, background, objective and justification	26/1/2026
Chapter 2: Research background	Retrieve the relevant information, system requirements and research gaps	7/2/2026
Chapter 3: Methodology	Defined the system development tools and requirements.	24/2/2026
Chapter 4: System Design	Mock up prototype, system architecture and data flow diagram	14/3/2026
Project I Portfolio	Submission	13/4/2026
Phase 3: FYP2		
Chapter 5: Development & Testing	Develop the main functions	26/6/2026
Chapter 6: System deployment	Deploy and test the system in real-world scenarios.	6/7/2026
Chapter 7: Project conclusion	Final summary and outcome of the project	1/8/2026

Figure 1.4: The project schedule for each phases

1.5 Project Team & Organization

This project is developed as an individual final year project under the guidance of a project supervisor. The student is responsible for system design, development, testing, and documentation. The supervisor provides academic guidance, technical advice, and feedback to ensure that the project meets academic requirements and follows proper development practices.

1.6 Chapter Summary and Evaluation

This chapter introduced the EthernaCare project, highlighting its shift from traditional monitoring to a **gamified wellness check-in system**. By replacing standard greeting messages with a **virtual pet interaction (petting a cat)**, the project aims to increase user engagement and provide a more intuitive "heartbeat signal" for elderly safety. The overall plan and objectives were defined to ensure this gamified approach effectively addresses the risks of delayed emergency discovery.

Chapter 2

Literature Review

2 Literature Review

This chapter examines previous research and existing systems related to elderly individuals who live alone and the growing concern of lonely deaths. It looks at demographic changes such as population ageing and the rise in independent living among older adults. It also reviews current technology solutions, including wearable emergency devices, smart home monitoring systems, and mobile safety applications. Through this review, the chapter highlights the strengths and weaknesses of existing approaches and clarifies how the proposed project addresses identified gaps and limitations.

2.1 Aging Population in Malaysia

2.1.1 Demographic Shift Toward an Ageing Society

Malaysia is moving toward an ageing nation as the percentage of citizens aged 60 and above continues to increase. The Department of Statistics Malaysia (*Current Population Estimates, Malaysia, 2023*, 2023) projects that older adults will make up around **15%** of the population by 2030. This change is mainly influenced by longer life expectancy and lower birth rates. The transition creates new challenges in healthcare planning, elderly care services, and community support systems.

2.1.2 Increase in Elderly Living Alone

The increase in elderly individuals living alone is closely linked to broader demographic and social transitions. According to the *World Report on Ageing and Health* by the World Health Organization (2015), although more generations within a family may now be alive at the same time due to increased life expectancy, these generations are increasingly living separately. In many countries, the proportion of older people living alone is rising dramatically. For example, in several European countries, more than 40% of women aged 65 and above live alone. Even in societies with strong traditions of multigenerational living, such as Japan and India, extended family households are becoming less common. Declining fertility rates mean there are fewer younger family members available to provide care, while changing gender roles have reduced the traditional caregiving capacity within families. Smaller household sizes may limit reciprocal care arrangements and increase the risk of poverty. Furthermore, older adults living alone may face higher risks of social isolation and suicide. Although many older people prefer to remain in their own homes and communities for as long as possible, these structural changes suggest

that traditional family-based care models are becoming less sustainable in the modern era (World Health Organization, 2015).

2.2 Technology concepts

2.2.1 Smartphone-Based Passive Monitoring

Smartphone-based monitoring systems use built-in sensors and activity logs to observe user behavior without requiring constant manual input (Wang, 2014). This passive approach allows the system to detect unusual patterns such as prolonged inactivity, which may indicate potential emergencies. Because the monitoring process runs automatically in the background, it reduces the need for continuous interaction and can operate without disturbing users.

2.2.2 Daily Check-In Monitoring Systems

Daily check-in monitoring systems are used in safety and well-being applications to verify that users are active through simple interactions with a mobile device. Instead of continuously collecting sensor data, gamified check-in systems utilize **virtual pets or interactive elements** to maintain user engagement. By requiring a tactile interaction like 'petting' an on-screen character, the system ensures the user is physically capable and responsive, serving as a more interactive form of passive monitoring (Rashidi & Mihailidis, 2013).

2.2.3 Wearable or Sensor Monitoring

Wearable and sensor-based monitoring systems are commonly used in elderly care technologies to continuously track health conditions and daily activities. These systems typically involve wearable devices such as smartwatches or sensor bands that collect physiological and movement data, including heart rate, walking patterns, and sleep activity. The collected data can be analyzed to identify abnormal patterns such as falls, prolonged inactivity, or sudden changes in physical behaviour (Patel et al., 2012) .

2.2.4 Duolingo and Behavioral Gamification

Modern gamification success stories, most notably **Duolingo**, demonstrate that habit formation is driven by emotional engagement and loss aversion. According to research on Duolingo’s growth strategy, the use of a persistent mascot (Duo the Owl) and a daily "streak" mechanic transforms a high-effort task into a "sticky" daily routine (Hasan, 2026). By quantifying

consistency, users feel a sense of ownership over their progress—a psychological phenomenon known as the **Endowment Effect**.

EthernaCare adapts these proven tactics by replacing standard check-in prompts with a virtual cat companion. By leveraging the emotional bond between the user and the digital pet, the system ensures high retention rates, as users are motivated to "check-in" to maintain their pet's well-being and their own reward progress.

2.3 Technology Acceptance Among the Elderly

2.3.1 Smartphone Accessibility Among Elderly Users

In recent years, smartphone ownership among older adults has increased significantly. According to the Pew Research Center (2021), smartphone adoption among individuals aged 65 and above has grown substantially compared to previous decades. The increased accessibility of smartphones provides opportunities to develop mobile-based solutions for elderly safety and monitoring. Since many elderly individuals already own smartphones, mobile applications can serve as cost-effective tools without requiring additional hardware (Pew Research Center, 2025).

2.3.2 Perceived Usefulness and Ease of Use

Recent studies show that perceived usefulness and ease of use remain strong predictors of technology adoption among older adults. Research on digital health and mobile monitoring systems indicates that elderly users are more likely to adopt applications when they believe the system improves their safety and supports independent living (Anderson et al., 2020). Confidence in smartphone usage and prior experience with digital tools also influence acceptance. When applications are perceived as complicated or unnecessary, adoption rates decrease.

2.3.3 Importance of User-Friendly Design

Usability plays a critical role in elderly technology adoption. Studies on mobile health applications emphasize that simple navigation, readable fonts, clear instructions, and minimal interaction steps significantly improve user satisfaction and long-term engagement (Dusseljee-Peute et al., 2018). Complex menus and small interface elements may discourage elderly users. Therefore, systems designed for older adults should prioritize clarity, accessibility, and reduced cognitive effort.

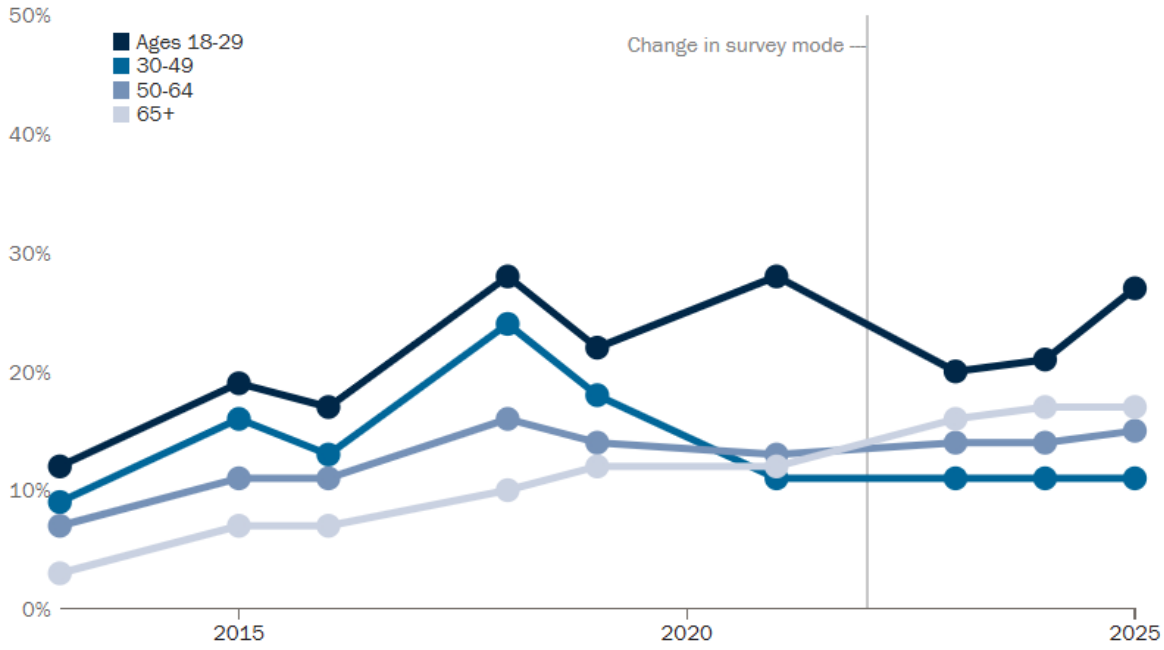


Figure 2.1: Percentages of U.S. adults who are smartphone dependent, by age

(Pew Research Center, 2025)

2.4 Similar Existing Systems in the market

2.4.1 Apple Emergency SOS

Apple Emergency SOS is a built-in smartphone feature that allows users to quickly contact emergency services and notify selected emergency contacts. The system can be activated using hardware button combinations, and it automatically shares the user's location with emergency responders and trusted contacts (Apple Inc., n.d.). The main feature is immediate emergency calling with real-time location transmission. It is integrated into iOS devices and depends on manual activation by the user.

This system is related to EthernaCare because both involve emergency alerts and contact notification. The difference is that EthernaCare uses a **daily greeting check-in mechanism** to confirm the user's status instead of requiring manual triggering. In addition, EthernaCare integrates post-death planning and digital document storage, which are not included in Apple Emergency SOS.

2.4.2 CarePredict

CarePredict is a wearable-based monitoring system designed for elderly individuals, particularly those in assisted living facilities. The system uses a wearable device equipped with sensors to monitor daily activities such as eating, sleeping, walking, and bathroom visits. It applies artificial intelligence and behavior pattern analysis to detect anomalies that may indicate health or safety risks (CarePredict, n.d.).

The main feature is continuous activity tracking through wearable sensors combined with AI-based behavioral analytics. The strength of the system lies in its detailed monitoring capabilities. However, it requires users to wear a dedicated device and is often implemented in institutional care settings.

EthernaCare shares the concept of behavior monitoring but differs by using smartphone-based passive detection rather than a wearable device. It is also designed for elderly individuals living independently at home rather than those in assisted living environments.

2.5 Limitation of the project

2.5.1 No Direct Medical Monitoring

The system does not measure heart rate, blood pressure, oxygen level, or other vital signs. It does not include fall detection using wearable sensors. It cannot diagnose medical conditions or confirm the cause of an emergency.

2.5.2 No Direct Integration with Hospitals or Emergency Services

The application does not automatically contact hospitals, ambulances, or police. Alerts are sent only to registered family members or emergency contacts. Further action depends on the response of those contacts.

2.5.3 No Smart Home or External Hardware Integration

The system does not connect to IoT devices, CCTV cameras, or home sensors. It does not require or support wearable tracking devices. Monitoring is limited to smartphone-based activity detection only.

2.6 Chapter Summary and Evaluation

This chapter analyzed the challenges of elderly isolation and the risks of delayed emergency response. It reviewed technology concepts including passive monitoring and daily check-ins, concluding that **gamification elements**—specifically interactive virtual companions—can bridge the gap between technical safety requirements and user acceptance. This review justifies the implementation of a tactile "petting" interaction as a reliable and less intrusive method of monitoring.

Chapter 3

Methodology and Requirements Analysis

3 Methodology and Requirements Analysis

This chapter describes the methodology and requirements analysis for the proposed system. It explains the development approach used in the project and identifies the stakeholders involved in the system. The chapter also presents the functional and non-functional requirements needed to support the system features. In addition, system models such as the use case diagram are introduced to illustrate how users interact with the system and how the main system functions are organized.

3.1 Methodology

3.1.1 Methodology Used: Agile

The project adopts the **Agile methodology**, which emphasizes iterative development, continuous feedback, and incremental improvements. Instead of completing the entire system at once, the system is developed in small functional modules such as daily check-in monitoring, emergency alerts, and digital planning features.

3.1.2 Methodology Phases

1. **Requirement Gathering** – Identify system requirements such as passive monitoring, alert triggering, and document storage.
2. **Design** – Create system architecture, UI wireframes, and database structure.
3. **Development** – Implement features module by module using Flutter and Supabase.
4. **Testing** – Conduct functional and usability testing for each module.
5. **Evaluation** – Review feedback and refine features before the next iteration.

3.1.3 Justification for Methodology

The Agile methodology is suitable for this project because the system consists of multiple independent modules, including activity monitoring, emergency alerts, AI chatbot guidance, and document storage. These modules can be developed and tested separately in iterative cycles. Additionally, system requirements may evolve after usability testing with potential users, especially elderly individuals, making a flexible development approach necessary. Iterative development helps reduce project risks by identifying issues early and allowing continuous improvements. It also enables critical features, such as emergency alert functions, to be tested and refined at an early stage of development.

3.2 Requirements Analysis

3.2.1 Stakeholders

The primary user is an elderly person or independent individual living alone. The user registers or signs in, completes first-login profile and consent setup, verifies phone numbers, taps Oren for daily check-ins, manages trusted contacts, configures inactivity thresholds, views rewards and weather, uses AI guidance, and records legacy-planning information.

Primary Trusted Contact

A trusted contact is an emergency recipient rather than a separate authenticated app role. The contact receives SMS follow-up when the user manually triggers an alert or reaches the configured inactivity escalation. One contact is designated as primary, and each stored contact number must be verified by SMS OTP before it can be saved.

System Administrator / Developer

The administrator or developer maintains the Flutter application, Supabase schema and policies, storage bucket, Edge Functions, OAuth providers, API secrets, logs, backups, and releases. There is currently no administrator dashboard exposed inside the end-user application.

3.2.2 Functional Requirements

Module 1: Authentication, Onboarding, and Profile

Users can register with validated email and password credentials or sign in through configured OAuth providers. New accounts complete mandatory profile details, accept the Terms and Conditions, choose Malaysian state and region values, select a blood type, configure inactivity settings, and verify the user phone number by SMS OTP. OAuth users are linked to a public profile row so that all Supabase-backed modules use the authenticated user ID consistently (Supabase, n.d.-a).

Module 2: Gamified Daily Check-In and Oren Care

Tapping Oren records at most one safety check-in per day, updates the history and streak, awards a daily token when eligible, and changes Oren mood and animation. Users can feed Oren, buy toys with Oren tokens, play with owned toys, hear interaction sounds, and return Oren to the default state after a short reaction.

Module 3: Inactivity Monitoring and Emergency Response

The configurable inactivity threshold represents one missed check-in window. Android WorkManager performs best-effort periodic checks when operating-system constraints allow (Android Developers, n.d.). The app shows reminders 1/3, 2/3, and 3/3. On the third reminder it records an alert and starts the configured escalation. Primary-contact escalation attempts SMS delivery; 999 remains a manual dialer action. A separate safe test counter sends a clearly labelled test SMS on test reminder 3.

Module 4: Trusted Contact and Location Management

Users can create, read, update, delete, and select one primary contact. Each account can store up to five contacts with validated name, relationship, international phone number, address, state, and region. Contact phone numbers require SMS OTP verification. Emergency alerts can store the device location and include a Google Maps link when location permission and positioning are available.

Module 5: Rewards and Oren Shop

The reward service calculates streaks from check-in history, synchronizes the reward catalogue, caches reward snapshots locally, displays the next reward on the dashboard, and supports virtual voucher-style rewards. Oren tokens are separate local gamification currency used for toys and interactions.

Module 6: Adaptive Malaysia Weather

The user selects a Malaysian state and region. The app requests the district or town forecast from the official data.gov.my Weather API and maps the returned Malay forecast summary to Oren backgrounds. If profile weather is unavailable, the app uses device coordinates with Open-Meteo. Successful weather responses are cached for 30 minutes (Government of Malaysia, n.d.; Open-Meteo, n.d.).

Module 7: Legacy Planning

Users can create, edit, read, and delete legacy notes; save funeral preferences; and upload, open, or remove private PDF or image documents up to 10 MB. Notes reject common credential and recovery-secret terms. When the owner explicitly enables Legacy Checking, the SMS-verified primary contact can use the owner's Legacy UID and a separate SMS OTP after 90 days without a check-in to view preferences and Legacy Notes without logging in. Secure documents are never released through this flow. EthernaCare stores completed documents but does not draft, validate, or notarize legal wills.

Module 8: AI Guidance

The AI Guidance screen sends the latest question and up to ten recent chat messages to a secured Supabase Edge Function. Gemini is the preferred provider, with optional OpenAI or custom-provider fallback. Conversation history is cached locally. The assistant gives general information only, does not diagnose or provide legal advice, and directs immediate Malaysian emergencies to 999 (Google AI for Developers, n.d.; Supabase, n.d.-b).

3.2.3 Functions not included

Funeral Service Booking and Payment System

While the application does not include a marketplace or transaction system. Users cannot browse specific funeral packages, service providers, or make payments for funeral services directly through the app.

Direct Communication with Service Providers

The application does not include a real-time chat or messaging system to connect users with funeral directors, lawyers, or religious organizations. The AI chatbot provides general information only and does not facilitate external business communication.

Medical and Biological Health Monitoring

The system relies solely on phone activity (screen interaction and app usage). It does not integrate with wearable devices (like smartwatches) to monitor heart rate, falls, or other biological vital signs. It is not a medical diagnostic tool.

Legal Will Drafting and Notarization

The application does not provide the legal tools to create, draft, or notarize a will. Users must obtain a legally binding will through traditional legal channels before uploading a copy to the app.

Direct Integration with Emergency Dispatch (999/911)

The system is designed to alert trusted contacts and family members first. It does not automatically dial or trigger a direct dispatch from government emergency services (Police or Ambulance) to prevent false alarms that could lead to legal liabilities or resource wastage. Except the user triggers the emergency call manually else the emergency services won't trigger.

3.2.4 Non-Functional Requirements

1. **Reliability**

- On Android, inactivity checks shall use best-effort WorkManager scheduling and continue after normal app restarts when operating-system constraints permit. The system shall not claim exact-time or uninterrupted 24/7 execution.
- Emergency alerts shall include a retry mechanism if the first notification attempt fails.
- User data (contacts, preferences, documents) shall not be lost during normal app updates.
- The system shall automatically resume monitoring after device restart.

2. Usability

- The application interface shall use clear fonts and simple navigation suitable for elderly users.
- Users shall be able to configure inactivity time thresholds easily.
- The emergency status shall be clearly displayed on the home screen.
- Manual SOS and 999 actions shall use confirmation or explicit user actions. Automatic primary-contact escalation shall occur only after three missed threshold windows.

3. Performance Efficiency

- The system shall run daily reminder and check-in tracking continuously without causing noticeable device lag.
- The system shall optimize battery usage to prevent excessive power consumption during background monitoring.

3.3 Use Case Diagram

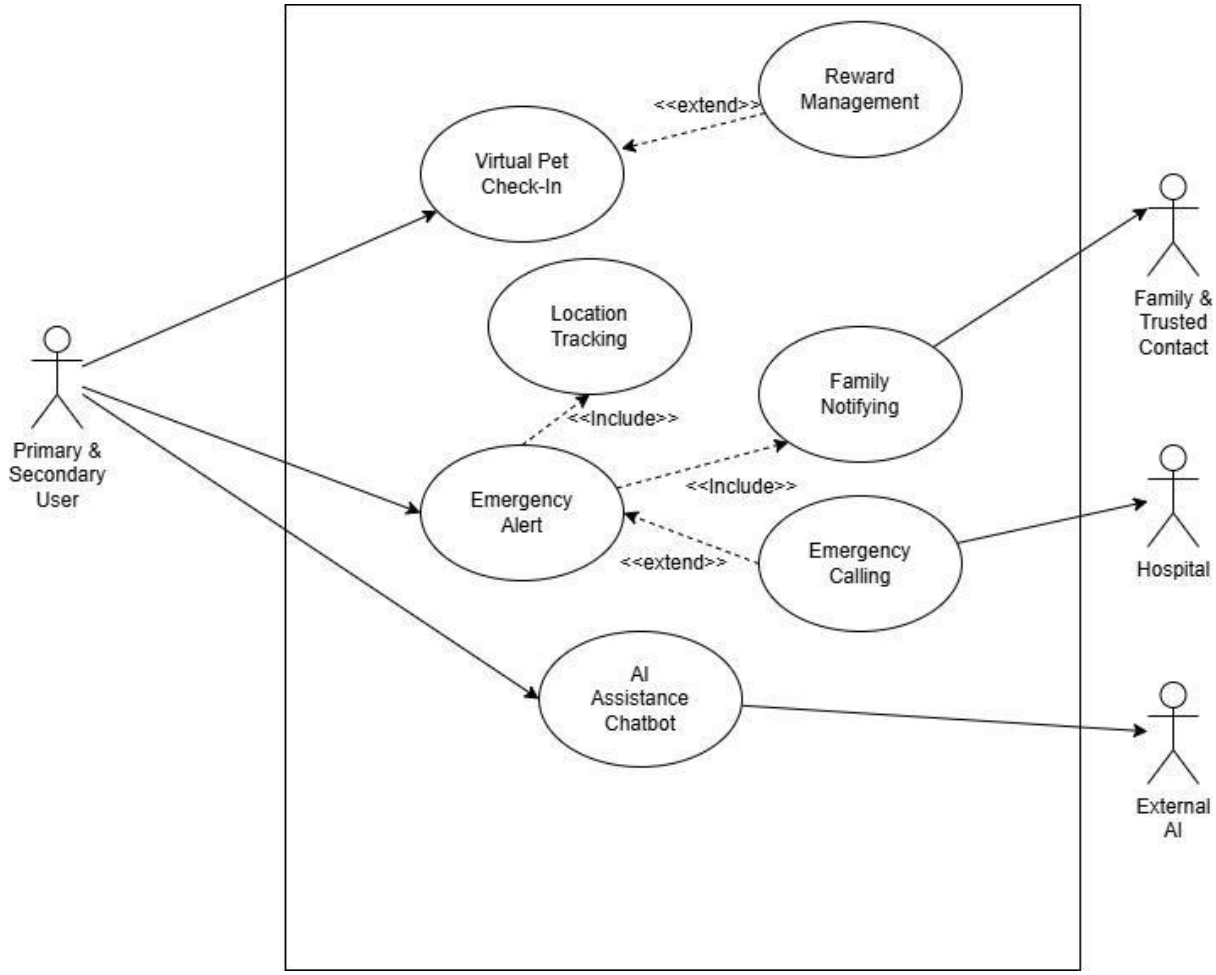


Figure 3.1: Use Case Diagram for each of stakeholders

3.3.1 Use Case Actors

Primary User - registers or signs in, completes onboarding, verifies a phone number, manages the profile and contacts, taps Oren to check in, uses Oren care and rewards, views weather, asks AI guidance, manages legacy planning, and triggers SOS.

Primary Trusted Contact - receives a verified, user-authorized emergency or test SMS. The contact does not log in to the current application.

External Services - Supabase provides authentication and protected data services; data.gov.my and Open-Meteo provide weather; Gemini provides AI content; Twilio or the Android SMS bridge provides SMS delivery; and device services provide location and notifications.

Main Use Cases

Daily Check-In - the user taps Oren; the system prevents duplicate daily records, updates history and rewards, resets the current inactivity cycle, and refreshes the dashboard.

Inactivity Escalation - the scheduler calculates elapsed threshold windows, records reminder counts, shows three notifications, and starts the configured escalation on the third reminder without automatically calling 999.

Contact Management - the user validates and verifies a contact number, stores up to five contacts, selects one primary contact atomically, and can edit or delete owned records.

Legacy Planning - the user manages funeral preferences, notes, and private documents under authenticated ownership policies.

AI Guidance - the user asks a question, receives provider or offline guidance, and retains recent local chat history.

3.4 Chapter Summary and Evaluation

This chapter detailed the Agile methodology and functional requirements for EthernaCare. Key requirements were identified, focusing on the **cat-petting check-in module** as the primary safety verification mechanism. The use case diagrams illustrate how this gamified interaction integrates with the **existing rewarding system**, where consistent "pet care" (check-ins) allows users to earn physical daily necessities. These requirements form the foundation for a system that balances safety, engagement, and practical incentives.

Chapter 4

System Design

4 System Design

This chapter presents the system design of the EthernaCare application. It illustrates how the system is structured and how its components interact to fulfill the required functionalities. The chapter includes the architecture design, class diagram, entity relationship diagram (ERD), and activity diagrams to provide a clear representation of the system's structure and behavior. These design models serve as a blueprint for the system implementation and ensure that the application is well-organized, scalable, and aligned with the defined requirements.

4.1 Architecture Diagram

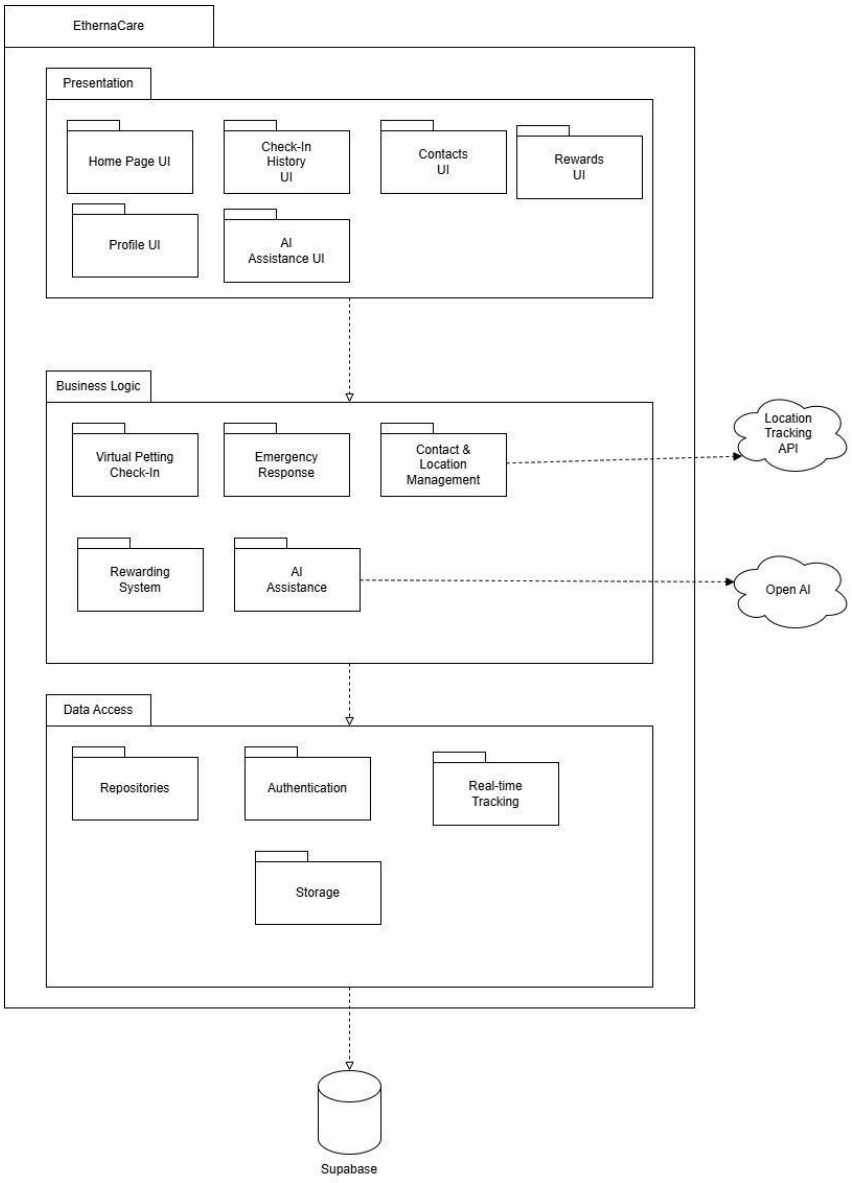


Figure 4.1: 3 Layer + 2 Tier Architecture diagram

4.1.1 Architecture Justification

EthernaCare follows a layered Flutter client and cloud-service architecture. The Presentation layer contains responsive screens and reusable widgets. Controllers and services coordinate validation, check-ins, inactivity policy, rewards, weather, AI, notifications, SMS, Oren care, and local caching. Repository classes isolate Supabase Auth, PostgreSQL, Storage, and Edge Function access. Models provide typed serialization, while Android background components invoke the inactivity service outside the foreground UI. This separation follows Flutter guidance that distinguishes UI responsibilities from repositories and services in the data layer (Flutter, n.d.).

Presentation Layer

The Presentation layer is responsible for handling all user interactions through interfaces such as the Home Page UI, Check-In History UI, Contacts UI, Rewards UI, Profile UI, and AI Assistance UI. This layer is designed to provide a simple, clear, and user-friendly experience, which is especially important for elderly users who may not be familiar with complex applications. By separating the user interface from the system logic, changes to the UI can be made without affecting the core functionality of the system. This improves usability, maintainability, and allows for easier future enhancements such as redesigning the interface or adding new features.

Business Logic Layer

The business and service layer applies application rules, including one check-in per day, reward synchronization, contact validation, primary-contact selection, three-stage inactivity escalation, safe test flows, weather caching, AI fallback behavior, and Oren state transitions. Controllers remain thin and delegate reusable work to services.

Data Access Layer

The Data Access layer contains repositories for authentication, users, check-ins, contacts, rewards, emergencies, legacy planning, and documents. Repositories use the authenticated Supabase session and RLS-protected tables. Edge Functions hold third-party secrets and integrate Gemini, Twilio, and phone OTP delivery without exposing provider credentials in the Flutter client (Supabase, n.d.-b; Supabase, n.d.-c).

4.1.2 Architecture Advantages

- **Scalability:** Individual layers in the architecture can be scaled independently to meet performance needs.
- **Flexibility:** Different technologies can be used within each layer without affecting others.
- **Maintainability:** Changes in one layer do not necessarily impact other layers, thus simplifying the maintenance. (Geeksforgeeks, 2025)

4.2 Activity Diagram

4.2.1 Login Process

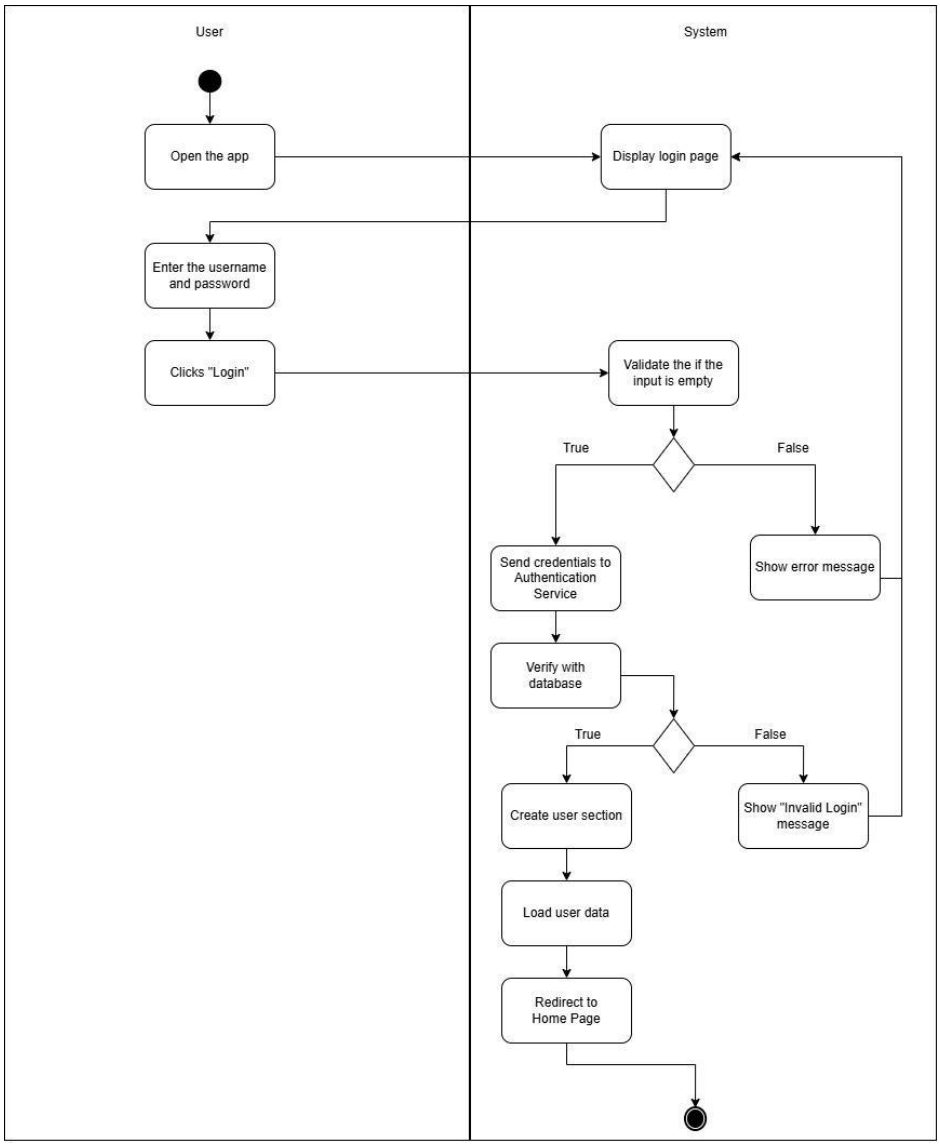


Figure 4.2.1: Activity Diagram show the login process of the EthernaCare

The login flow validates email and password inputs or starts a configured OAuth provider. Supabase creates a JWT-backed session and the app ensures that an associated public user profile exists. A new account is routed through mandatory profile, Terms and Conditions, user-phone OTP, and primary-contact OTP setup before reaching the dashboard. Returning users with complete setup proceed directly to the home screen. Friendly error dialogs replace raw provider exceptions.

4.2.2 Virtual Petting Check-In

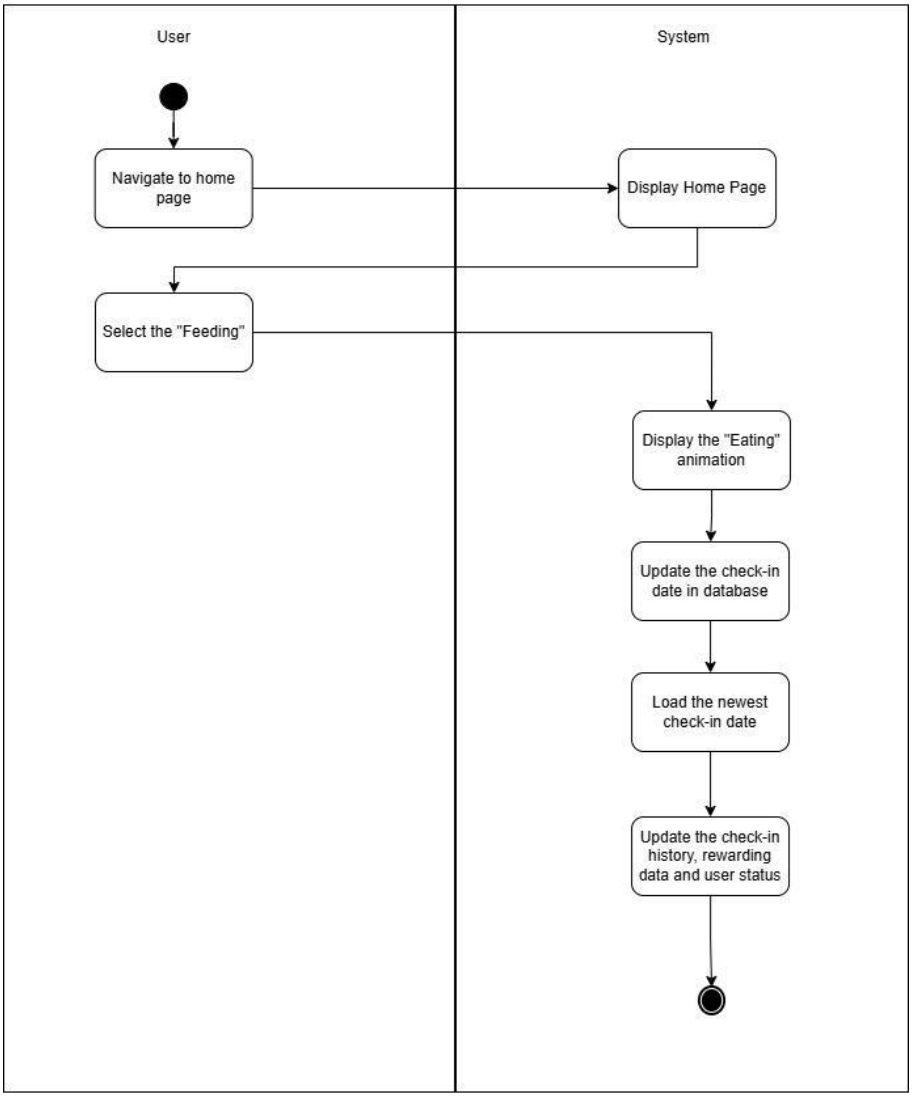


Figure 4.2.2: Activity Diagram show the process of Virtual Petting

The user taps Oren directly rather than pressing a separate Pet button. The repository checks for an existing record in the current local day and inserts a new Supabase check-in only when one does not exist. The service then refreshes the dashboard, streak, rewards, Oren tokens, mood, sound, and interaction animation. A completed check-in also makes any earlier local inactivity reminder state obsolete.

4.2.3 Emergency Alert

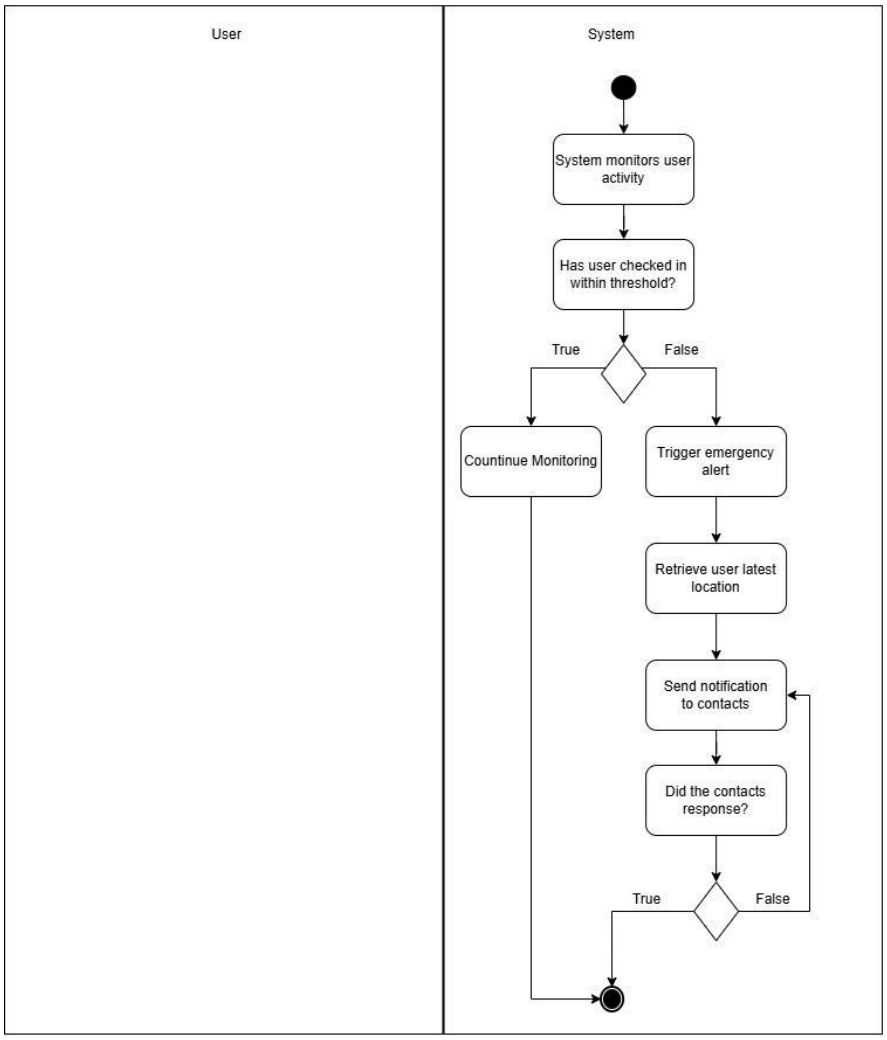


Figure 4.2.3: Activity Diagram show the process of Emergency Alert

The emergency process supports manual SOS and automatic inactivity escalation. For inactivity, the service divides elapsed time since the latest check-in by the user's configured threshold. It shows one local notification per missed window and records a visible 0/3 to 3/3 dashboard counter. Reminder 3 starts escalation. If the profile target is the primary trusted contact, the app retrieves that contact, records an emergency alert, attempts Android SMS, and otherwise queues a Twilio Edge Function delivery with retry information. Location is attached when available. If the user selects 999, the inactivity event records a critical notice but does not automatically dial emergency services. Test alerts are stored separately and cannot suppress or duplicate real alerts.

4.3 ERD Relationship Diagram

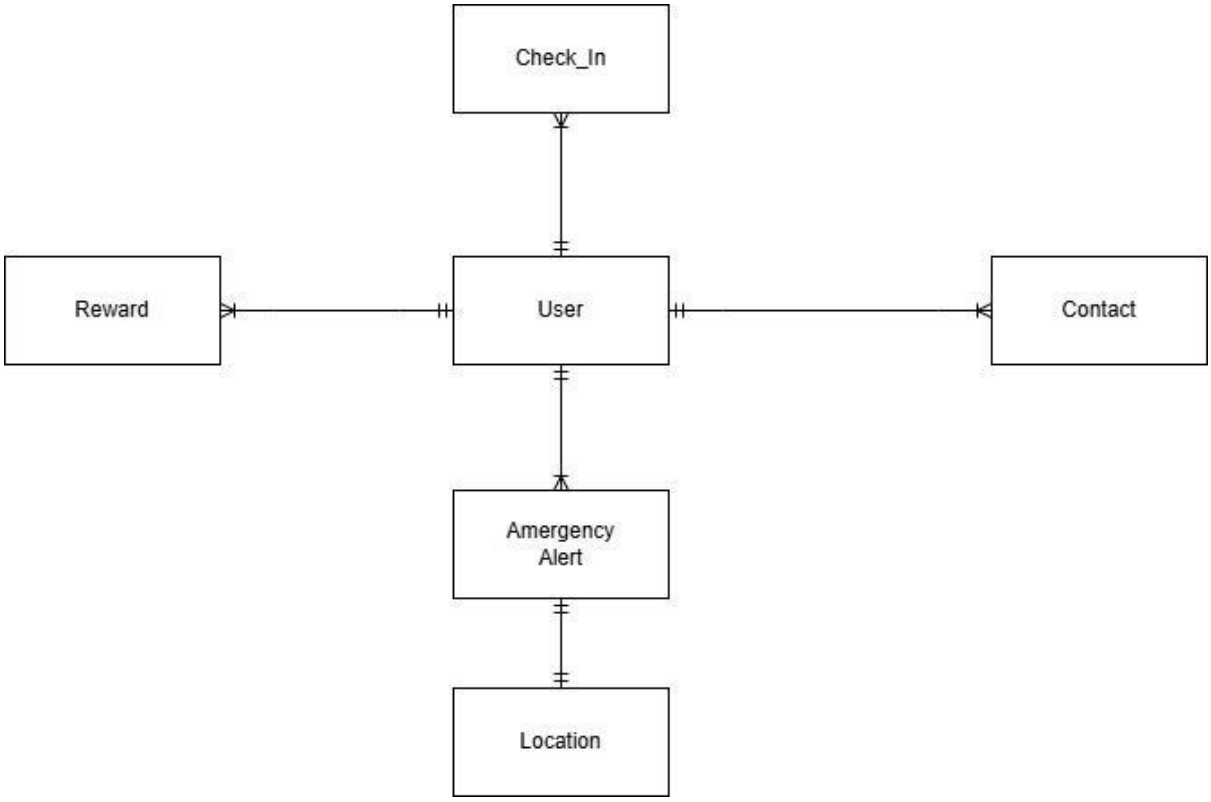


Figure 4.3: ERD diagram that describes the relationship between the entities

4.3.1 Entities attributes

User

Attributes: id, name, email identity, phone, phone_verified_at, address, address_state, address_region, blood_type, inactivity_threshold, emergency_escalation_target, terms_version, terms_accepted_at, and profile_completed_at. Age is not collected.

Contact

Attributes: id, user_id, name, relationship, phone, phone_verified_at, address, address_state, address_region, and is_primary. One user owns up to five contacts and one primary contact.

Check-In

Attributes: id, user_id, checkin_time, and status. A user owns many daily activity records.

Emergency Alert and Location

Emergency alerts contain id, user_id, triggered_time, and status. Optional location records contain alert_id, latitude, longitude, and timestamp.

Reward and Reward Catalogue

User rewards contain streak, reward type/code, status, earned and redeemed timestamps. The catalogue stores title, sponsor, milestone, kind, voucher value, version, and active state.

Legacy Planning

Funeral preferences are one-to-one with the user. Legacy notes and secure documents are one-to-many. Documents store private storage paths and upload timestamps.

SMS Delivery and Phone Verification

The emergency delivery outbox stores contact, message, provider, attempts, status, errors, and processed time. OTP records store only a code hash, purpose, expiry, consumption state, and attempt count; successful verification records are retained separately.

4.3.2 ERD Relationships

1. **User → Contact (1:M)**

   One user can have multiple contacts, but each contact belongs to only one user. This allows multiple family members or trusted contacts to be notified during emergencies.

2. **User → Check-In (1:M)**

   One user can perform many check-ins over time. Each check-in records user activity to detect inactivity and ensure safety monitoring.

3. **User → Emergency Alert (1:M)**

   One user can trigger multiple emergency alerts. Each alert represents a separate incident and is stored for tracking purposes.

4. **Emergency Alert → Location (1:1)**

   Each emergency alert is linked to one location record. This ensures accurate location tracking during emergencies.

5. **User → Reward (1:M)**

   One user can earn multiple rewards. This supports the gamification system based on consistent check-ins.

6. **User → Document (1:M)**

   One user can store multiple documents (e.g., will, funeral preferences). Each document belongs to a single user to maintain privacy and ownership.

4.4 Security Handling

Authentication and Access Control

Supabase Auth supports email/password and configured OAuth providers. Authenticated sessions use JWTs. A database trigger creates the matching public profile for email and OAuth accounts so every repository uses the same authenticated user ID (Supabase, n.d.-a).

Row Level Security and Private Storage

RLS is enabled on exposed user-owned tables. Policies compare auth.uid() with id or user_id for select, insert, update, and delete operations. The legacy-documents bucket is private and storage policies restrict paths to the authenticated user folder (Supabase, n.d.-c).

Phone Verification and Input Validation

Six-digit OTP codes are generated server-side, stored as hashes with expiry and attempt limits, and delivered through an Edge Function. Database triggers reject changed profile or contact phone numbers that do not have a recent successful verification. Client and database validation also enforce name, password, phone, address, contact-count, and primary-contact rules.

Secrets and Secure Communication

The Flutter client communicates through HTTPS. Gemini, Twilio, SMS worker, and OTP secrets are stored as Supabase Edge Function environment secrets rather than in source code. Edge Functions validate authenticated users or a worker secret before privileged operations (Supabase, n.d.-b).

4.5 UI Prototype

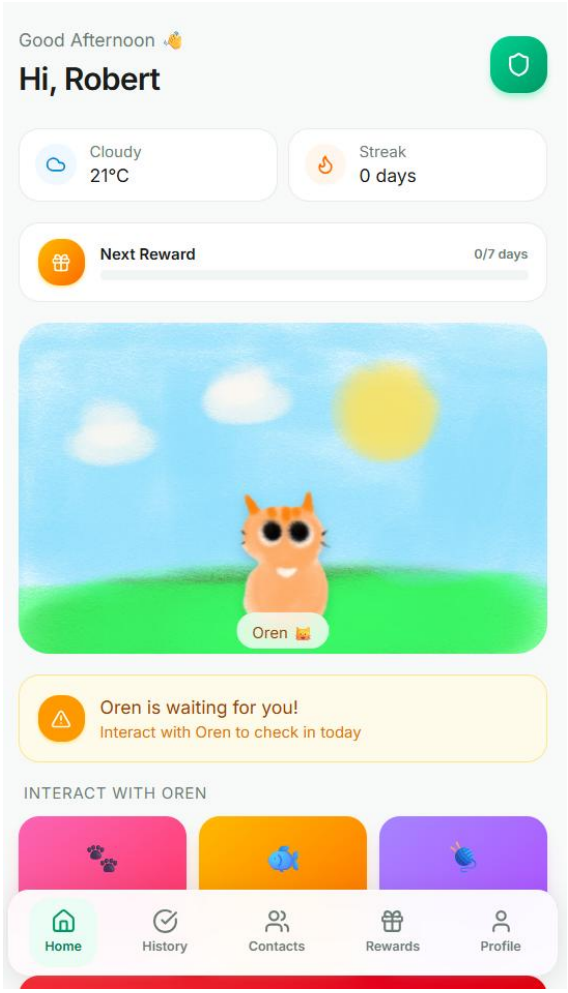


Figure 4.4.1: Home Page

The current home page uses a responsive premium dashboard. Oren appears at the top as the direct check-in target and changes pixel-art mood, energy, interaction, selected toy, and weather background. Feed Fish and Play are presented together below the Oren scene, while the token balance and shop controls support toy purchase and selection. The Safety Monitor is positioned lower on the page and shows real inactivity reminder progress, a separate three-step test action, emergency state, SOS, and SMS testing. Contextual guidance, scroll cues, and safe-area spacing support small Android screens.

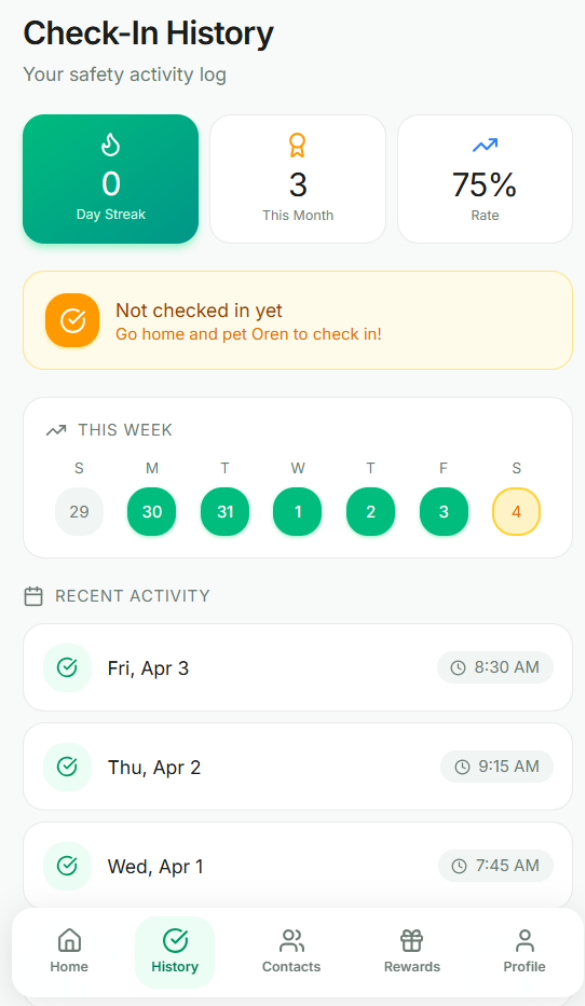


Figure 4.4.2: Check-In History Page

The Check-in History page provides a transparent and structured record of the user's daily interactions, serving as a visual "health diary" of their activity. It presents a chronological list of every successful "petting" event, displaying specific dates and timestamps to confirm when the "heartbeat signal" was sent to the system. This interface allows both the user and their caregivers to monitor consistency, highlighting daily streaks and any missed days that might indicate a change in routine. By centralizing this data, the page reinforces the habit-forming nature of the gamified system while offering peace of mind through a clear, verifiable history of the user's well-being and safety status.

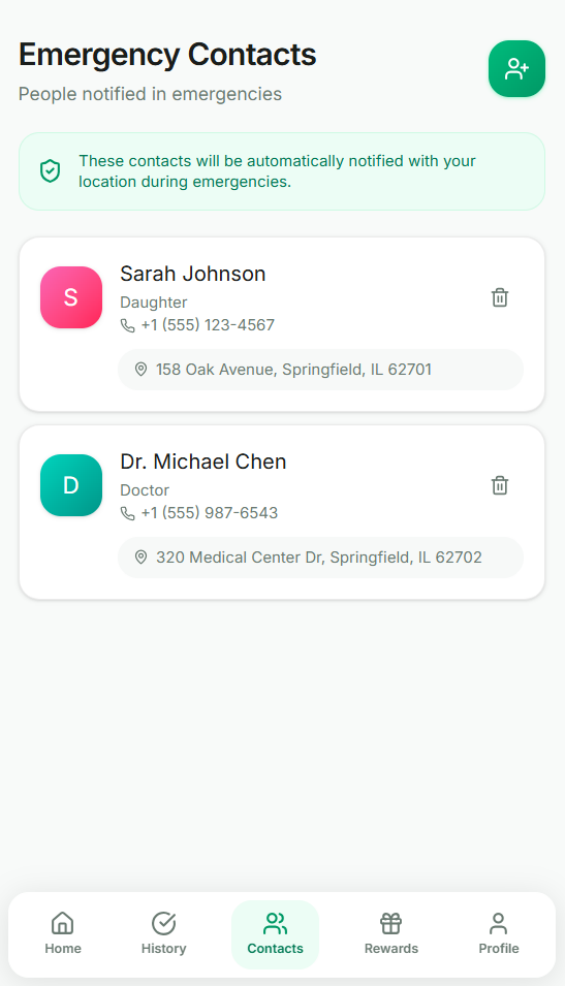


Figure 4.4.3: Emergency Contacts Page

The Emergency Contacts page lists up to five owned contacts with name, relationship, verified international phone number, address, state, and region. A star action changes the primary contact through an atomic database function, while edit and delete operations refresh the interface immediately. Adding or changing a phone number requires SMS OTP verification.

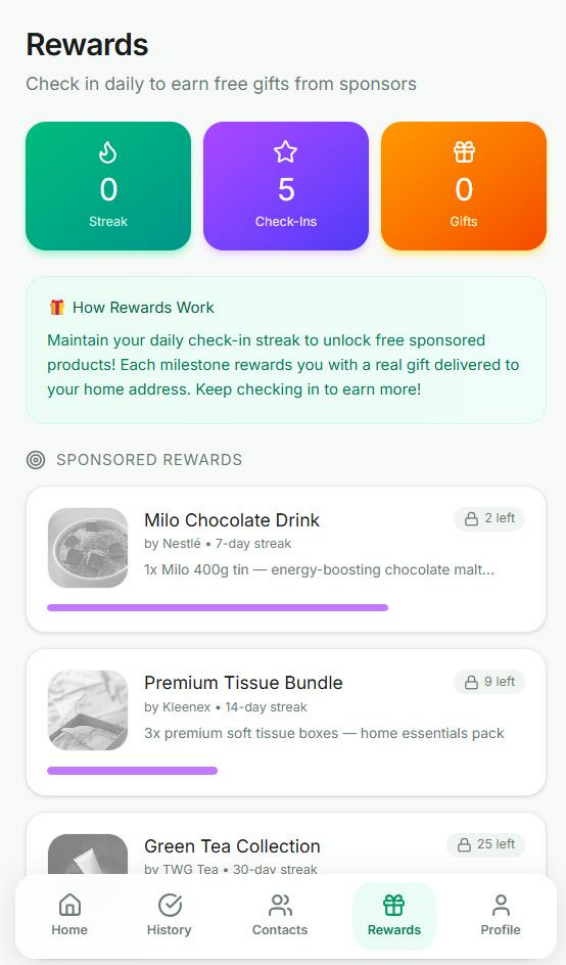


Figure 4.4.4: Rewards Page

The Rewards page combines server-backed streak rewards and locally cached reward catalogue data with Oren tokens and toys. The dashboard shows the next milestone reward, while the Oren shop allows owned toys to be purchased and used. Reward examples include virtual sponsor vouchers rather than guaranteed physical inventory.

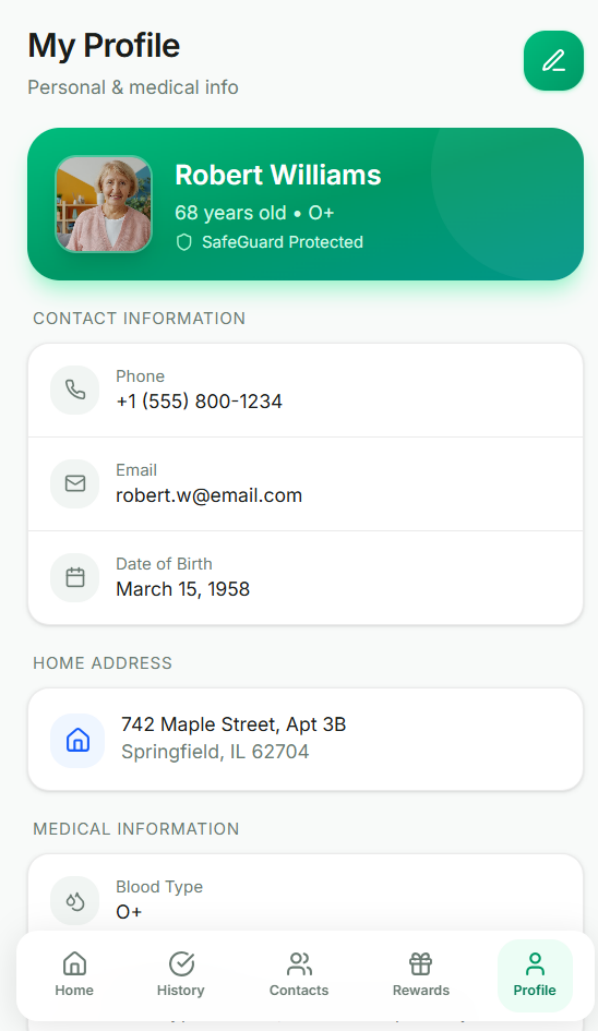


Figure 4.4.5: Profile Page

The Profile page displays name, verified phone, home address, Malaysian state and region, blood type, inactivity threshold, and escalation target. Age has been removed. Editing uses a full-page responsive form, country calling-code selection, validation, blood-type selection, and OTP verification when the phone number changes. The page provides a copyable Legacy UID, contextual guidance, AI Guidance, and Legacy Planning access.

4.6 Chapter Summary and Evaluation

This chapter presented the overall system design of the EthernaCare application, including the architecture, diagrams, and security considerations. The system adopts a three-tier architecture consisting of the Presentation, Business Logic, and Data Access layers to ensure a clear separation of concerns and improve maintainability. Various design models such as the class diagram, entity relationship diagram (ERD), and activity diagrams were developed to illustrate the system structure, data relationships, and process flows, including login and emergency alert functionalities. In addition, appropriate design patterns and security handling techniques were incorporated to enhance system reliability, scalability, and data protection. Overall, this chapter provides a comprehensive blueprint that guides the implementation of the system in the next phase.

Chapter 5

Implementation

5 Implementation

5.1 Introduction

This chapter explains how EthernaCare was translated from the requirements and system design into a working Flutter application. It follows the prescribed implementation structure by describing the development tools, implemented modules, representative code, sample screens, functional testing, non-functional testing, and the final module-based test plan.

Implementation was incremental. Each module was developed around a service or repository boundary, connected to the responsive presentation layer, and then checked with automated Flutter tests and manual walkthroughs. High-risk behavior such as inactivity escalation, SMS delivery, phone verification, private documents, Legacy Checking, and emergency calling was kept explicit and auditable.

5.2 System Implementation

EthernaCare uses a layered mobile-and-cloud architecture. Flutter widgets form the presentation layer; controllers and services apply reusable application rules; repositories isolate Supabase Auth, PostgreSQL, Storage, and Edge Function calls; typed models serialize data; and Android WorkManager invokes best-effort inactivity checks outside the foreground interface. SharedPreferences supplies per-user local caching for dashboard, weather, chat, rewards, Oren care, and reminder state.

5.2.1 Development Tools and Technologies

Flutter and Dart

Flutter 3 with Dart implements the Android, Windows, and web-capable interface from one codebase. Material 3 components, responsive constraints, safe areas, scrolling forms, semantic labels, and reusable widgets support consistent behavior across compact and wide viewports. The pubspec uses Dart SDK 3.10.4 or later and declares packages for Supabase, Riverpod, HTTP, location, notifications, WorkManager, caching, file selection, audio, and URL launching.

Supabase and PostgreSQL

Supabase provides email/password and OAuth authentication, PostgreSQL tables, Row Level Security, private Storage, remote procedure calls, and Edge Functions. SQL files in the repository are idempotent where practical so schema repairs, phone verification, document storage, legacy access, contact constraints, and OAuth profile bootstrap can be audited and applied independently (Supabase, n.d.-a; Supabase, n.d.-b; Supabase, n.d.-c).

Android and External Services

Android WorkManager performs a connected periodic task every 15 minutes when operating-system constraints allow. Local notifications present reminder stages. Device location can be attached to an alert, and direct SMS is attempted only with permission. Twilio is the server-side SMS alternative. The official Malaysia weather API is preferred for the selected region, Open-Meteo is the fallback, and Gemini is the preferred AI provider (Android Developers, n.d.; Government of Malaysia, n.d.; Google AI for Developers, n.d.; Open-Meteo, n.d.; Twilio, n.d.).

Development and Verification Utilities

Android Studio, the Android SDK, Git, PowerShell, the Supabase CLI through npx, Flutter Test, and the recommended Flutter lints support development and verification. Word and PDF rendering are used to maintain the report artefacts. API credentials are stored as Supabase project secrets and are not committed to the Flutter source.

5.2.2 Modules and Code Snippets

Authentication, Onboarding, and Profile Module

AuthRepository encapsulates registration, email sign-in, signup-code verification, resend verification, Google OAuth, and sign-out. AuthGate routes authenticated users through mandatory profile completion, Terms and Conditions, phone OTP verification, primary-contact setup, and the first-login tutorial. An authentication trigger and backfill create the matching public.users row for both email and OAuth identities so all repositories use the same authenticated UUID.

```
final response = await client.auth.signInWithPassword(email: email, password: password);

await userRepository.createProfileIfMissing(userId: user.id, email: user.email);
```

Daily Check-In, Oren Care, and Rewards Module

Oren is the direct daily check-in control. CheckinRepository first searches the current local calendar day and inserts only when no row exists. After a successful or already-existing check-in, the dashboard, streak, reward snapshot, Oren token balance, mood, energy, sound, and selected-toy reaction are refreshed. OrenCareService stores tokens, owned toys, selected toy, mood, energy, and daily reward dates under a user-specific cache key, so signing out and signing back in does not reset the shop state.

```
if (existing.isNotEmpty) return false;

await      client.from('checkins').insert({'user_id':      userId,      'checkin_time':
now.toIso8601String(), 'status': 'active'});
```

Inactivity Monitoring and Emergency Response Module

InactivityService converts elapsed time into missed threshold windows. Reminder 1 and reminder 2 notify the user. Reminder 3 records an emergency event and starts the configured escalation without automatically calling Malaysia's 999 service. EmergencyService obtains the primary contact, records location when available, attempts direct-device SMS, and otherwise creates or processes a server delivery. A separate test counter uses clearly labelled messages and cannot silently become a real emergency event.

```
missedCheckIns = elapsed.inSeconds ~/ Duration(hours: threshold).inSeconds;

if (missedCheckIns >= 3) await emergencyService.triggerEmergencyDetailed(allow999Dialer:
false, sendAutomatedSms: true);
```

Trusted Contacts, OTP, and Location Module

Users manage up to five contacts through full-page add and edit forms. Validation covers name, relationship, international phone, address, state, region, duplicate numbers, and primary-contact rules. A changed number must have a recent phone_verifications record before a database trigger accepts it. Contact ownership is enforced with RLS, while an atomic RPC and unique partial index preserve one

primary contact. SOS and inactivity alerts use the primary contact and include a Google Maps link when location permission is available.

```
create policy "contacts_select_own" on public.contacts for select

to authenticated using ((select auth.uid()) = user_id);
```

Weather, AI Guidance, and Local Cache Module

WeatherService first requests the selected Malaysian region from data.gov.my, maps Malay and English summaries to clear, cloudy, rain, and thunderstorm scenes, and caches successful data for 30 minutes. Device-coordinate Open-Meteo data is used only as a fallback. AiService sends the latest question and limited local conversation history to a secured Edge Function and returns a safety-oriented offline answer when the provider is unavailable.

Legacy Planning, Secure Documents, and Legacy Checking Module

DocumentService loads funeral preferences, Legacy Notes, secure document metadata, and the owner's Legacy Checking consent together. Notes support CRUD but reject common credential terms and secret-value patterns. File upload accepts PDF, JPG, JPEG, and PNG up to 10 MB, validates the extension and binary signature, stores the object in a private user folder, and opens it through a short-lived signed URL. Deletion removes both metadata and the private object.

```
if (fileSize > maxDocumentBytes) throw StateError('Document must not exceed 10 MB.');

if (!_matchesDocumentSignature(extension, bytes)) throw StateError('The file content does
not match its extension.');
```

Legacy Checking is intentionally separate from ordinary authentication. The owner copies a Legacy UID from Profile and explicitly enables access. After at least 90 days without check-in activity, the verified primary contact can request a dedicated SMS code and view only funeral preferences and Legacy Notes for a ten-minute foreground session. Secure documents, account credentials, and general profile data are excluded. The SQL and Edge Function source are implemented locally; cloud deployment and end-to-end SMS verification remain required before production use.

```
export const legacyInactivityDays = 90;

if (Date.now() - lastActivityMs < legacyInactivityDays * 86400000) denyLegacyAccess();
```

Responsive Guidance and Error Handling

Contextual information buttons open a reusable scrollable guidance sheet. The Profile guide explains tabs, Oren, tokens, contacts, SOS, and status colours; Legacy Planning and Legacy Check guides explain privacy and access conditions. Raw Supabase and provider exceptions are converted into task-specific dialogs, while layouts use stable dimensions, full-page forms, scrolling, tooltips, and responsive text constraints.

5.2.3 Sample Screens

The following screens demonstrate the major interaction modules. Figures 5.1 to 5.4 are representative captures from the application report build. Figure 5.5 is retained as an earlier profile prototype to document the UI evaluation that led to the current full-page editor, Malaysian address fields, copyable Legacy UID, and removal of age and date-of-birth collection.

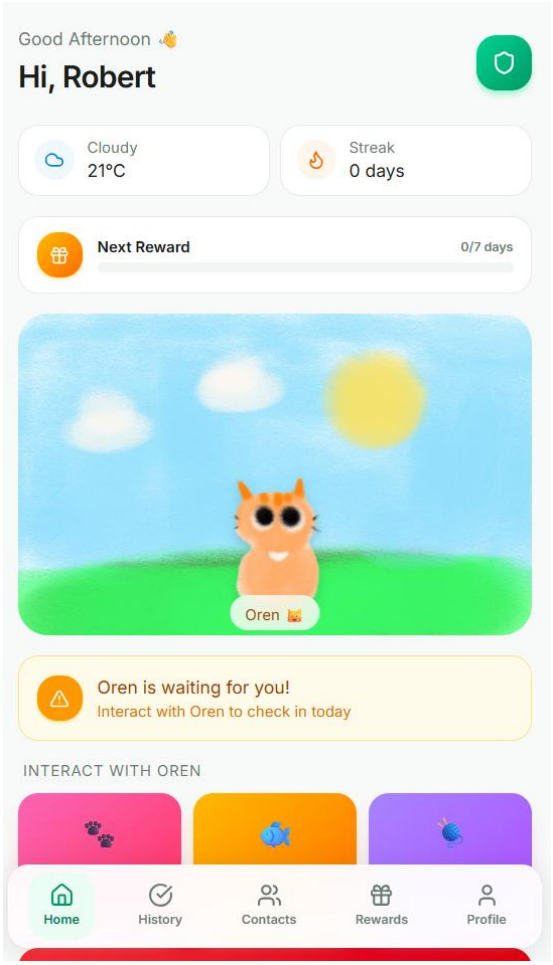


Figure 5.1: Home and Oren daily check-in module

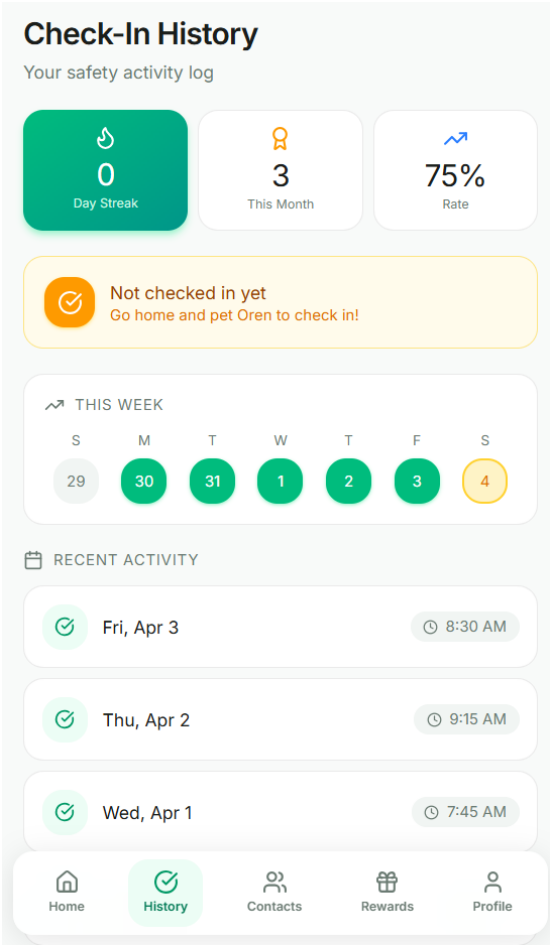


Figure 5.2: Check-in History module

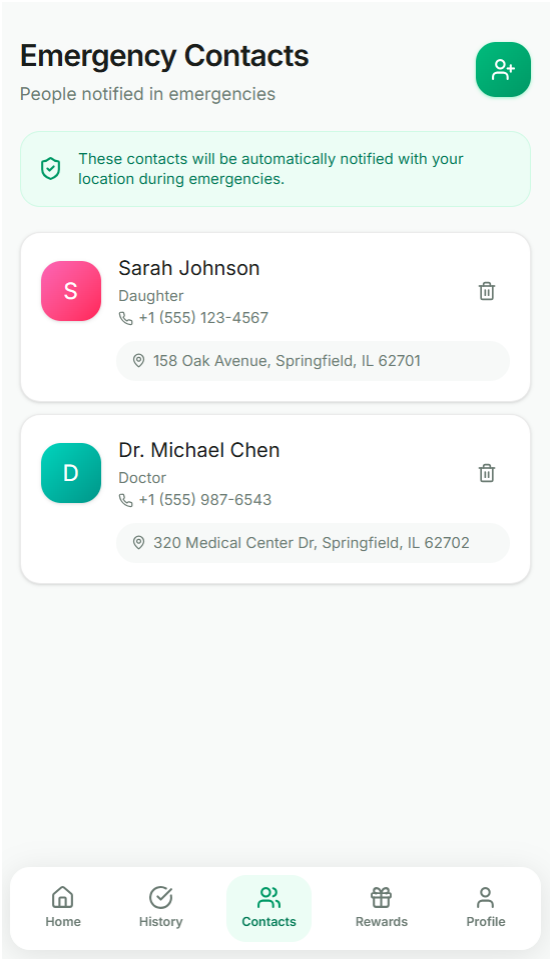


Figure 5.3: Trusted Contacts module

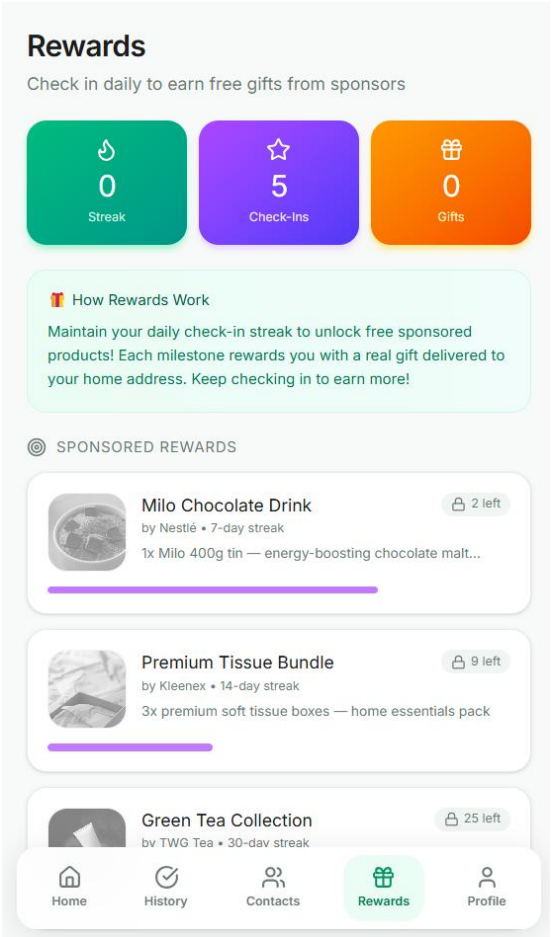


Figure 5.4: Rewards module

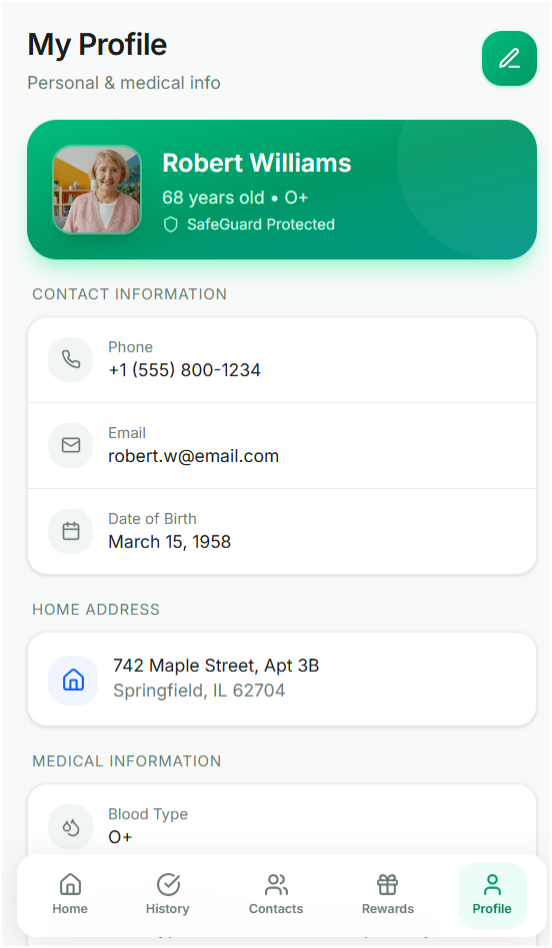


Figure 5.5: Earlier Profile prototype used during UI evaluation

The prototype reward wording and profile sample data shown in Figures 5.4 and 5.5 are not production promises or current data fields. The current rewards are virtual catalogue examples, and the current Profile does not collect age or date of birth.

5.3 Testing Strategies

Testing was performed continuously during Agile iterations. Automated tests verify deterministic logic and widget behavior, while manual integration and system walkthroughs cover flows that depend on Supabase, Android permissions, background scheduling, OAuth, SMS, location, weather, and AI services. The current Flutter suite contains 34 passing tests: 23 logic or model tests and 11 widget tests.

5.3.1 Functional Testing

Unit Testing

Unit tests validate inactivity boundaries, the three-step reminder cycle, streak calculation, reward and Oren serialization, per-user cache preservation, weather mapping, validation limits, onboarding rules, funeral-preference serialization, Legacy Note security, document file signatures and size limits, Legacy

Checking payload filtering, AI fallback behavior, and chat-history serialization. These tests isolate predictable rules without requiring a live provider.

Widget and Component Testing

Widget tests render the tutorial, responsive guidance sheet, bounded action rows, shared controls, Oren check-in and energy states, Add Contact validation, and Edit Profile validation. A 360 by 640 viewport is included for the guidance sheet to detect overflow and ensure that confirmation controls remain reachable.

Integration Testing

Repository and service boundaries were exercised through development builds against the configured Supabase project, including authentication, profile loading, contacts, check-ins, rewards, weather, and document metadata. Provider-dependent flows require valid remote migrations, deployed Edge Functions, Android permissions, and secrets. A dedicated automated integration_test suite is not yet included, so OAuth, Twilio delivery, phone OTP, Legacy Checking OTP, Gemini, location, and WorkManager behavior are listed separately as deployment-environment tests rather than reported as automated passes.

System Testing

System walkthroughs followed the main user journey: account access, first-login setup, profile editing, contact verification, Oren check-in, history refresh, rewards, weather, inactivity reminder testing, AI guidance, Legacy Planning, and file validation. The walkthroughs exposed and corrected blank post-login routing, stale OAuth profile data, small dialog forms, setState asynchronous callbacks, weather scenes that repeatedly appeared rainy, local Oren state reset, and inconsistent emergency presentation.

User Acceptance Testing

Developer-supervised acceptance checks were performed iteratively using direct user feedback on screenshots and working builds. Feedback resulted in full-page contact and profile forms, simpler Oren check-in, clearer region labels, scroll cues, consistent emergency controls, distinct Oren reactions, contextual guidance, and safer Legacy Notes. This is formative acceptance testing; a formal study with elderly participants, measured task completion, and accessibility instruments remains future work.

5.3.2 Non-Functional Testing

Reliability Testing

The 24, 48, and 72-hour boundary test confirms reminder counts of 1, 2, and 3 for a 24-hour threshold. Duplicate reminders and duplicate daily check-ins are suppressed, Oren state is preserved per user, and provider calls use retry or fallback behavior where implemented. WorkManager scheduling is best-effort, so exact execution timing, force-stop recovery, and restart behavior must be verified on physical Android devices rather than claimed as continuous 24/7 monitoring.

Usability and Accessibility Testing

Compact widget tests check overflow, while manual review checks safe areas, scrolling, touch targets, labels, tooltips, contrast, information hierarchy, and full-page input forms. The first-login tutorial and contextual guides reduce reliance on prior knowledge of Oren or safety terminology. Formal accessibility audits, screen-reader studies, and elderly-user task measurements are not yet complete.

Performance Efficiency Testing

Cache-first loading reduces repeated Supabase and weather calls, weather data is retained for 30 minutes, AI history is limited, and the background task runs at the platform minimum periodic interval rather than continuously. Development builds were checked for responsive interaction and absence of obvious lag. Formal CPU, memory, battery, network, and load benchmarks have not yet been conducted and remain a release-readiness activity.

Security Testing

Security-oriented tests and review cover validated authentication inputs, hashed and expiring OTP records, recent-phone-verification triggers, RLS ownership checks, private storage paths, signed document URLs, file magic-byte validation, Legacy Note credential rejection, dedicated Legacy access OTPs, release-field filtering, foreground timeout, and audit events. Production assurance still depends on applying the supplied SQL, deploying the intended Edge Functions, rotating exposed secrets, and reviewing Supabase policies in the deployed project.

Compatibility and Maintainability Testing

The same widgets are exercised at compact phone dimensions and compile for Flutter's supported targets, while Android-specific behavior is isolated behind platform components. Models, services, repositories, and screens are separated so external providers and UI layouts can change independently. The automated architecture and serialization checks reduce regression risk, but a broader physical-device and OS-version matrix is still required.

5.4 Test Plan and Test Cases

The test plan below is arranged by functional module as required by the supplied Chapter 5 guide. Pass indicates a reproducible automated test in the current suite. Configuration required identifies a valid test case whose result depends on remote deployment, credentials, a physical device, or a consented SMS recipient and therefore must not be presented as an automated pass.

ID	Module	Test scenario	Test steps / data	Expected result	Status
TC01	Authentication	Validate email/password and OAuth screen states	Enter invalid and valid credentials; inspect validation and routing	Invalid input is blocked; a valid session routes through setup	Pass
TC02	Onboarding/Profile	Enforce mandatory first-login setup	Leave required values empty, then complete profile, terms and phone state	Incomplete setup remains blocked; complete setup can continue	Pass
TC03	Check-In/Oren	Prevent duplicate daily check-ins	Tap Oren twice on the same local day	Only one daily record is eligible; the UI refreshes without duplication	Pass
TC04	Oren Care	Restore per-user shop and interaction state	Serialize tokens, toys, selected toy, mood and energy; reload the same user	The same user receives the saved Oren state after reload	Pass
TC05	Inactivity	Calculate three missed threshold windows	Evaluate elapsed times at 24, 48 and 72 hours for a 24-hour threshold	Reminder counts are 1/3, 2/3 and 3/3 respectively	Pass

ID	Module	Test scenario	Test steps / data	Expected result	Status
TC06	Reminder Test	Keep testing separate from real emergencies	Trigger the dashboard test action three times	The third action starts a clearly labelled test SMS flow only	Pass
TC07	Contacts	Validate contact data and primary-contact rules	Test names, phone digits, addresses, duplicates, limit and primary selection	Invalid data is rejected and only one contact is primary	Pass
TC08	Phone OTP	Verify profile and contact phone delivery	Deploy functions and secrets; request and submit a code on a consented phone	A valid code creates verification; invalid or expired codes fail	Configuration required
TC09	Rewards	Calculate streaks and restore reward snapshots	Load dated check-ins, reward catalogue and cached token data	Streak, next reward, tokens and owned items are consistent	Pass
TC10	Weather	Map forecast summaries to Oren scenes	Provide clear, cloudy, rain, thunderstorm and night forecast samples	Each summary selects the intended scene without defaulting to rain	Pass
TC11	Legacy Preferences	Serialize and restore funeral preferences	Save religion, service type, authorized contact and wishes, then reload	All selected and entered values are preserved	Pass
TC12	Legacy Notes	Reject credentials and support safe note CRUD	Try password/recovery terms, then create, edit and delete a normal note	Secret-like content is blocked; safe notes complete CRUD	Pass
TC13	Secure Documents	Validate uploaded document size and signature	Test valid PDF/PNG/JPEG plus oversized and mismatched files	Only supported files up to 10 MB with matching signatures are accepted	Pass
TC14	Legacy Checking	Release only authorized legacy fields	Build a release payload containing preferences, notes and private fields	Only preferences and notes remain; documents and profile data are excluded	Pass
TC15	Guidance/UI	Keep contextual help reachable on a compact screen	Render guidance and action rows at 360 by 640 pixels	Content scrolls without overflow and controls remain reachable	Pass
TC16	AI Guidance	Restore history and provide a safe offline fallback	Serialize recent messages and simulate unavailable AI service	History restores and general fallback guidance is returned	Pass
TC17	External Services	Verify live provider and Android integrations	Test Google OAuth, Twilio SMS, Gemini, location and WorkManager on deployment	Configured services complete with logs, permissions and delivery evidence	Integration setup required

The matrix demonstrates broad automated coverage of core rules and responsive components, while clearly separating external integration work. This distinction is important because a passing model or

widget test cannot prove Twilio delivery, Google OAuth redirect configuration, Android background timing, or remote RLS deployment.

5.5 Chapter Summary and Evaluation

This chapter implemented the required Flutter and Supabase modules, documented representative code and screens, and evaluated both functional and non-functional testing. Thirty-four automated tests pass for deterministic logic and widget behavior. External OAuth, SMS, OTP, AI, location, background execution, and deployed-policy checks remain explicitly identified as integration work. The implementation therefore satisfies the principal software requirements while avoiding unsupported claims about third-party delivery or exact-time emergency monitoring.

Chapter 6

Conclusion

6 Conclusion

6.1 Summary

EthernaCare was developed as an accessible safety and legacy-planning application for elderly people and independent individuals who live alone. It uses a friendly virtual cat as a daily activity signal, applies measured inactivity escalation, protects user-owned cloud records, and presents supporting weather, rewards, AI guidance, contacts, and legacy-planning functions in one Flutter application.

6.1.1 Project Objectives and Achievement

The first objective was to use a gamified virtual-cat interaction to confirm that a user is active. This objective was achieved through direct Oren check-in, one-record-per-day protection, history, token rewards, mood and energy changes, toys, and a three-stage inactivity reminder process. The experience is less clinical than a conventional monitoring form while still producing an auditable activity record.

The second objective was to help users prepare post-death information. This objective was achieved through funeral-preference fields, selection of an authorized existing contact, safe Legacy Note CRUD, and private PDF or image document storage. Legacy Checking extends the design by allowing an explicitly authorized, verified primary contact to view only preferences and notes after 90 days of inactivity. The cloud deployment of its dedicated OTP functions remains a required production step.

The third objective was to provide basic AI-assisted information. This objective was achieved through AI Guidance with recent chat history, a secured provider function, safety boundaries, and an offline fallback. The system provides general guidance rather than medical, legal, or emergency diagnosis. Collectively, these objectives support SDG 3 by combining well-being, safety awareness, dignity, and family preparedness.

6.1.2 Overall System or Product Developed

The resulting product is a responsive Flutter application with five primary navigation areas: Home, History, Contacts, Rewards, and Profile. Home contains Oren, the selected regional weather scene, care actions, token and shop controls, the next reward, and a lower Safety Monitor. Additional full-page flows cover onboarding, profile editing, contact verification, AI Guidance, Legacy Planning, secure documents, and public Legacy Checking.

6.1.3 Methods and Tools Used

Agile iteration was used to refine requirements and repair issues discovered through screenshots, working builds, schema errors, provider responses, and user feedback. Flutter and Dart implemented the client; Riverpod, services, repositories, and typed models separated concerns; Supabase supplied Auth, PostgreSQL, RLS, Storage, RPCs, and Edge Functions; SharedPreferences supplied local per-user caching; and Android notifications, location, WorkManager, and SMS bridges supported safety behavior. Flutter Test, SQL audits, Git, Android Studio, Supabase CLI, PowerShell, Word, and PDF review supported implementation and verification.

6.1.4 Outcome and System Performances

The deterministic application logic and responsive components currently pass 34 automated tests, comprising 23 logic or model tests and 11 widget tests. Cache-first reads and a 30-minute weather cache reduce repeated network calls, while bounded histories and periodic background work limit unnecessary

processing. Compact-screen tests confirm reachable scrollable guidance and stable controls. Formal battery, memory, network-load, accessibility, and elderly-participant benchmarks have not yet been completed; provider-dependent performance also remains subject to deployment configuration and third-party availability.

6.2 Achievements & Contribution

6.2.1 System Achievements

The project implements email and Google-capable authentication, OAuth profile bootstrap, mandatory first-login setup, verified profile and contact phones, trusted-contact CRUD, direct Oren check-in, immediate history refresh, rewards, persistent Oren tokens and toys, region-based weather, local reminders, inactivity escalation, location-assisted emergency records, safe SMS testing, AI history and fallback, funeral preferences, credential-aware notes, private document upload, copyable Legacy UID, and contextual guidance. Important safety distinctions are preserved: reminder testing is separate from real alerts, secure documents are not shared by Legacy Checking, and 999 calling always requires an explicit user action.

6.2.2 Contributions

The main contribution is the combination of a low-effort virtual-pet habit with an explicit safety signal and privacy-aware personal planning. Oren provides an emotionally approachable reason to return each day, while the Safety Monitor keeps the meaning of missed activity visible. Per-user cache separation, verified trusted contacts, controlled legacy release, private signed document access, friendly error presentation, and honest fallback states contribute practical safeguards that are often missing from simple check-in demonstrations.

6.3 Limitations and Future Improvements

Android background work is best-effort and can be delayed by battery optimization, connectivity, manufacturer policy, or force-stop behavior; the web build has no equivalent periodic scheduler. SMS, phone OTP, Google OAuth, Gemini, weather, and location depend on external credentials, quotas, billing, device permissions, and provider availability. Twilio trial restrictions can also limit recipients. The local cache is device-specific, and the application is not a medical monitor: it does not detect falls, vital signs, or unconsciousness.

Future work should move inactivity evaluation to a reliable server scheduler, add push notification and SMS delivery receipts, deploy and exercise all Legacy Checking functions, add automated integration and end-to-end tests, measure battery and network use, conduct formal accessibility and elderly-user acceptance studies, add Malay and Chinese language support, provide secure account recovery, and evaluate a consent-based caregiver portal. Any legal-document feature should be developed only with professional legal review.

6.4 Issues and Solutions

6.4.1 Technical Issues

Google OAuth users initially authenticated without a matching public profile, which caused Supabase-backed pages to appear stale or empty. An auth.users trigger and backfill now create the public.users row and repositories use the authenticated UUID consistently. Schema-cache errors such as a missing

documents.uploaded_at column were addressed with idempotent SQL repair files and schema audits. Contact validation conflicts were resolved with aligned columns, triggers, RLS, and an atomic primary-contact RPC.

Several UI defects were also corrected. Small profile and contact dialogs became scrollable full-page forms; asynchronous setState callbacks were separated from awaited work; a blank post-login route was isolated behind explicit loading and setup states; weather selection now prioritizes the saved Malaysian region instead of defaulting to a rain scene; Oren state uses user-specific cache keys; and emergency controls use consistent messages and confirmation paths. OTP authentication errors are now surfaced as configuration problems rather than silent failures.

6.4.2 Project Management Issues

The project scope expanded as safety, privacy, accessibility, and deployment details became clearer. A modular backlog and small idempotent SQL files allowed urgent defects to be repaired without rewriting unrelated modules. The report was repeatedly reconciled with the actual repository so unfinished provider deployment, external quotas, formal benchmarking, and legal limitations were not overstated. Future iterations should define integration acceptance criteria and secret-management tasks earlier in the schedule.

6.4.3 Team Dynamics and Collaboration Issues

This is an individual final-year project, so group-team conflict and task-allocation issues are not applicable. Collaboration instead occurred through supervisor guidance, direct user feedback, platform documentation, and iterative review of builds and report artefacts. Recording decisions and maintaining clear module boundaries helped keep those inputs traceable.

6.4.4 Learning and Skill Development Challenges

The project required growth across Flutter responsive layout, asynchronous state, Android lifecycle constraints, Supabase Auth and RLS, PostgreSQL triggers and RPCs, private Storage, Edge Functions, OAuth redirects, SMS credentials, secure OTP design, file-signature validation, API fallback behavior, automated testing, and academic documentation. The most important learning was that a successful local widget test is different from proof of remote policy, provider delivery, or background execution on a physical device.

6.4.5 Overall Lessons Learnt and Future Improvements

Safety-critical wording and behavior must be conservative, test modes must remain visibly separate, private data release must use explicit authorization and minimum necessary fields, and external dependencies require observable configuration checks. Future planning should reserve time for physical-device testing, provider dashboards, elderly-participant evaluation, deployment logs, privacy review, and regression automation. These lessons provide a realistic route from the current final-year project to a more dependable production service.

6.5 Conclusion

EthernaCare demonstrates that a simple daily virtual-pet interaction can become a broader safety and preparedness system without disguising its limitations. The project achieved its principal software objectives and produced a modular, tested application foundation. Production use still requires completed provider deployment, physical-device verification, formal user evaluation, operational monitoring, and continued privacy and accessibility review.

References

References

Anderson, J. E., Ross, A. J., Macrae, C., & Wiig, S. (2020). Defining adaptive capacity in healthcare: A new framework for researching resilient performance. Applied Ergonomics, 87, 103111. https://doi.org/10.1016/j.apergo.2020.103111

Android Developers. (n.d.). WorkManager API reference. Retrieved July 11, 2026, from https://developer.android.com/reference/androidx/work/WorkManager

Apple Inc. (n.d.). Use Emergency SOS via satellite on your iPhone. Retrieved February 24, 2026, from https://support.apple.com/en-us/101573

CarePredict. (n.d.). Senior living safety and care optimization. Retrieved February 24, 2026, from https://www.carepredict.com/

Czaja, S. J., et al. (2006). Factors predicting the use of technology: Findings from the Center for Research and Education on Aging and Technology Enhancement. Psychology and Aging, 21(2), 333-352. https://doi.org/10.1037/0882-7974.21.2.333

Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Quarterly, 13(3), 319-340. https://doi.org/10.2307/249008

Department of Statistics Malaysia. (2023). Current population estimates, Malaysia, 2023. https://www.dosm.gov.my

Department of Statistics Malaysia. (2024). Marriage and divorce statistics, Malaysia, 2024. https://www.dosm.gov.my/portal-main/release-content/marriage-and-divorce-2024

Department of Statistics Malaysia. (2025). Migration survey report, Malaysia, 2024. https://www.dosm.gov.my/portal-main/release-content/migration-survey-report-malaysia-2024

Dusseljee-Peute, L. W. P., Jaspers, M. W. M., & Wildenbos, G. A. (2018). Aging barriers influencing mobile health usability for older adults: A literature-based framework (MOLD-US). International Journal of Medical Informatics, 114, 66-75.

Flutter. (n.d.). Guide to app architecture. Retrieved July 11, 2026, from https://docs.flutter.dev/app-architecture/guide

Google AI for Developers. (n.d.). Gemini API: Generating content. Retrieved July 11, 2026, from https://ai.google.dev/api/generate-content

Government of Malaysia. (n.d.). Weather API. Malaysia's Official Open API. Retrieved July 11, 2026, from https://developer.data.gov.my/realtime-api/weather

Lee, C.-W., & Chuang, H.-M. (2021). Design of a seniors and Alzheimer's disease caring service platform. BMC Medical Informatics and Decision Making, 21(Suppl 10), 273. https://doi.org/10.1186/s12911-021-01626-3

Melchiorre, M. G., D'Amen, B., Quattrini, S., Lamura, G., Socci, M., & Tchounwou, P. B. (2022). Health emergencies, falls, and use of communication technologies by older people with functional and social frailty. International Journal of Environmental Research and Public Health, 19(22), 15150. https://pmc.ncbi.nlm.nih.gov/articles/PMC9691100/

Open-Meteo. (n.d.). Weather Forecast API. Retrieved July 11, 2026, from https://open-meteo.com/en/docs

Patel, S., Park, H., Bonato, P., Chan, L., & Rodgers, M. (2012). A review of wearable sensors and systems with application in rehabilitation. Journal of NeuroEngineering and Rehabilitation, 9, 21. https://doi.org/10.1186/1743-0003-9-21

Rashidi, P., & Mihailidis, A. (2013). A survey on ambient-assisted living tools for older adults. IEEE Journal of Biomedical and Health Informatics, 17(3), 579-590. https://doi.org/10.1109/JBHI.2012.2234129

Supabase. (n.d.-a). Auth. Retrieved July 11, 2026, from https://supabase.com/docs/guides/auth

Supabase. (n.d.-b). Edge Functions. Retrieved July 11, 2026, from https://supabase.com/docs/guides/functions

Supabase. (n.d.-c). Row Level Security. Retrieved July 11, 2026, from https://supabase.com/docs/guides/database/postgres/row-level-security

Twilio. (n.d.). Messages resource. Retrieved July 11, 2026, from https://www.twilio.com/docs/messaging/api/message-resource

United Nations, Department of Economic and Social Affairs, Population Division. (2022). World population prospects 2022: Summary of results. https://www.un.org/development/desa/pd/

Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. MIS Quarterly, 27(3), 425-478. https://doi.org/10.2307/30036540

Vlaev, I., King, D., Darzi, A., & Dolan, P. (2019). Changing health behaviors using financial incentives: A review from behavioral economics. BMC Public Health, 19, 1059. https://pmc.ncbi.nlm.nih.gov/articles/PMC6686221/

Wang, R., et al. (2014). StudentLife: Assessing mental health, academic performance and behavioral trends of college students using smartphones. Proceedings of UbiComp 2014, 3-14. https://doi.org/10.1145/2632048.2632054

Wang, Z., Wang, Y., Zeng, Y., Su, J., & Li, Z. (2025). An investigation into the acceptance of intelligent care systems: An extended technology acceptance model. Scientific Reports, 15, 17912. https://doi.org/10.1038/s41598-025-02746-w

World Health Organization. (2015). World report on ageing and health. https://www.who.int/publications/i/item/9789241565042

Appendices

APPENDIX A User Guide

Account and First Login

Register with a valid email and strong password or use a configured OAuth provider. Complete required profile details, accept the Terms and Conditions, verify the user phone, and add and verify a primary trusted contact.

Daily Use

Open Home and tap Oren once each day to check in. Review History for timestamps, Rewards for streak milestones, and the dashboard for weather, next reward, and inactivity state. Use Feed and owned toys for Oren interactions.

Safety Functions

Maintain an accurate primary contact and inactivity threshold. The real counter reaches three reminders before configured escalation. Use the Safe reminder test only with the contact's consent. For immediate danger, explicitly open the 999 dialer and place the call.

Profile, AI, and Legacy Planning

Edit state, region, blood type, threshold, and escalation target from Profile. Use the copy button beside Legacy UID only when preparing an authorized Legacy Checking request. AI Guidance provides general information only. Legacy Planning stores preferences, safe notes, and completed private documents; it does not create legal documents. Legacy Checking must be enabled by the owner, is limited to the verified primary contact after 90 days without a check-in, and never releases secure documents.

Contextual Guidance

Use the information buttons in Profile, Legacy Planning, and Legacy Check to open the scrollable guidance sheet. It explains Oren check-in, tokens, shop items, inactivity status, contacts, SOS, Legacy UID privacy, and the conditions for legacy access.

APPENDIX B Developer Guide

Development Requirements

Install Flutter/Dart, Android Studio or Android SDK tools, Git, and Node.js for npx-based Supabase CLI commands. Restore packages with flutter pub get and verify with flutter test.

Supabase Setup

Create the Supabase project, configure the app URL and anon key, run the SQL migration and RLS files, create the private legacy-documents bucket, configure OAuth redirect URLs, deploy Edge Functions, and set Gemini, Twilio, OTP, and worker secrets. Apply the Legacy Checking consent, audit, release, and OTP schema before deploying its dedicated functions. Never commit service-role or provider secrets.

Build and Verification

Run schema_audit.sql, execute the Flutter tests, build an Android APK or release bundle, install on a physical device, grant permissions, and test email/OAuth, OTP, contacts, check-in, rewards, weather, AI, legacy documents, location, notifications, and the safe three-reminder SMS cycle.

Maintenance

Monitor Supabase Auth, database, Storage, Edge Function logs, SMS outbox failures, API quotas, and crash reports. Apply idempotent migrations before releasing code that expects new columns. Rotate leaked credentials immediately and keep provider billing and sender verification current.